

Fatima Jinnah Women University

Department of Computer Science

Lab 03: Working with Git History, Stashing, and Reverting Commits

Lab Submission Report

Course	DevOps (CS-649 Cloud DevOps)
Instructor	Sir Waqas Saleem
Student Name	Kiran Siddique
Registration No.	2023-BCS-052
Section	A
GitHub Username	KiranSiddique
Submission Repository	DevOps (Labs/Lab03)
Date of Submission	19 October 2026

Table of Contents

Part 1: Commands Used in This Lab **3**

Part 2: Step-by-Step Procedure with Screenshots **6**

Task 1: Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase) 6

Task 2: Creating and Resolving Merge Conflicts Manually 12

Task 3: Managing Ignored Files with .gitignore and Removing Tracked Files 16

Task 4: Create Temporary Changes and Use git stash 20

Task 5: Checkout a Specific Commit Using git log 22

Task 6: Resetting Commits (Soft vs Hard Reset) 24

Task 7: Amending the Last Commit 31

Task 8: Reverting a Commit (Safe Undo on Remote Branch) 32

Task 9: Force Push (With Caution) 34

Task 10: Running Gitea in GitHub Codespaces via Docker Compose 36

Task 11: Creating a GitHub Pages Portfolio Site 41

Part 1: Commands Used in This Lab

The table below lists the Git, Docker and Ubuntu commands used in each task, in the order they were run.

Task	Commands

Task 1	`git fetch`
	`git log --oneline origin/main -2`
	`nano README.md`
	`cat README.md`
	`git add README.md`
	`git commit -m "Local update to README"`
	`git status`
	`git push origin main`
	`git pull --no-rebase origin main`
	`git commit --no-edit`
	`git log --oneline --graph -4`
	`git push -u origin main`
	`git remote set-url origin git@github.com:KiranSiddique/DevOps_Lab3_Practice.git`
	`git commit -m "Second local update"`
	`git pull --rebase origin main`
	`git log --oneline --graph -5`

Task 2	`nano README.md`
	`cat README.md`
	`git add README.md`
	`git commit -m "Local conflicting change"`
	`git push -u origin main`
	`git pull --rebase origin main`
	`git rebase --continue`

Task	Commands

Task 3	`mkdir textfiles`
	`echo "File A content" > textfiles/a.txt`
	`echo "File B content" > textfiles/b.txt`
	`echo "File C content" > textfiles/c.txt`
	`git add .`
	`git commit -m "Added textfiles directory with three files"`
	`git push origin main`
	`echo "textfiles/*" > .gitignore`
	`git add .gitignore`
	`git commit -m "Added .gitignore to ignore textfiles directory"`
	`git rm -r --cached textfiles`
	`git commit -m "Removed tracked textfiles directory"`
	`ls textfiles`

Task 4	`git checkout -b feature-branch`
	`echo "Feature branch line 1" >> README.md`
	`echo "Feature branch line 2" >> README.md`
	`git add README.md`
	`git commit -m "Changes the README.md file"`
	`echo "Uncommitted change line 1" >> README.md`
	`echo "Uncommitted change line 2" >> README.md`
	`git checkout main`
	`git stash`
	`git checkout feature-branch`
	`git status`
	`git stash pop`

Task 5	`git log --oneline`
	`git stash`
	`git checkout 6c32eb1`
	`git checkout main`

Task 6	`echo "Test line 1" >> README.md`
	`git add .`
	`git commit -m "Added test line"`
	`echo "Test line 2" >> README.md`
	`git commit -m "Second test commit"`
	`git log --oneline`
	`cat README.md`
	`git reset --soft HEAD~1`
	`git status`
	`git commit -m "Second Test commit"`
	`git reset --hard HEAD~1`

Task	Commands

Task 7	`echo "log fix" >> README.md`
	`git add .`
	`git commit -m "Fix log message"`
	`echo "notes line" > notes.txt`
	`git commit --amend --no-edit`
	`git show --stat HEAD`

Task 8	`echo "temporary text" >> temp.txt`
	`git add .`
	`git commit -m "Added temporary text file"`
	`git push origin main`
	`git log --oneline -3`
	`git revert --no-edit <hash>`
	`git log --oneline -4`

Task 9	`git checkout -b test-force`
	`echo "force test line" >> force.txt`
	`git add .`
	`git commit -m "Force test commit"`
	`git push origin test-force`
	`git reset --hard HEAD~1`
	`git push origin test-force --force`

Task 10	`git checkout codespace`
	`docker compose down`
	`docker compose up -d`
	`docker compose ps`

Task 11	`git clone git@github.com:KiranSiddique/Portfolio.git`
	`cd Portfolio`
	`ls -lh`
	`git remote -v`
	`git log --oneline -3`

Part 2: Step-by-Step Procedure with Screenshots

Each step explains what was done and why, followed by the screenshot taken for that step. Screenshots are listed with their figure number and caption directly below them.

Task 1: Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase)

Step 1: Edited README.md directly on GitHub by adding the line 'This line was added remotely from GitHub.' and committed it with the message 'Remote update to README'. This creates a new commit on the remote main branch that the local repository does not have yet.

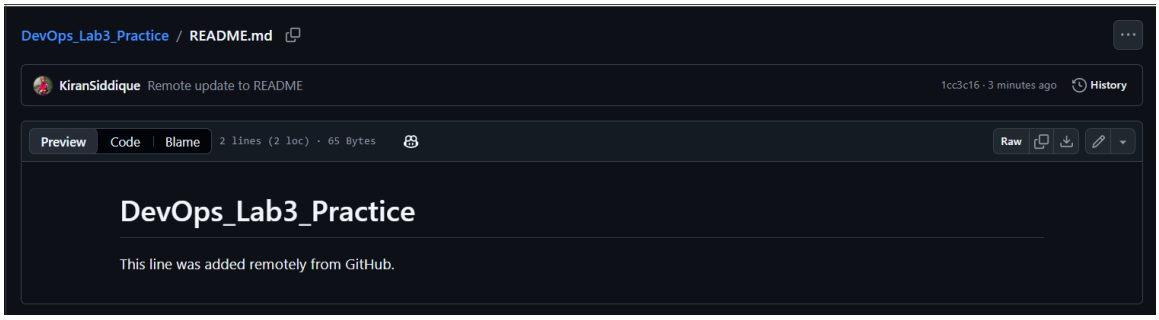


Figure 1.1: README.md edited on GitHub and committed (Remote update to README).

Step 2: Fetched the remote changes to confirm the new remote commit. Then edited README.md locally without running git pull, adding the line 'This line was added locally.' below the title. The change was staged and committed. The git status output shows that the local and remote branches have diverged, each with one different commit.

```
git fetch
git log --oneline origin/main -2
nano README.md
cat README.md
git add README.md
git commit -m "Local update to README"
git status
```

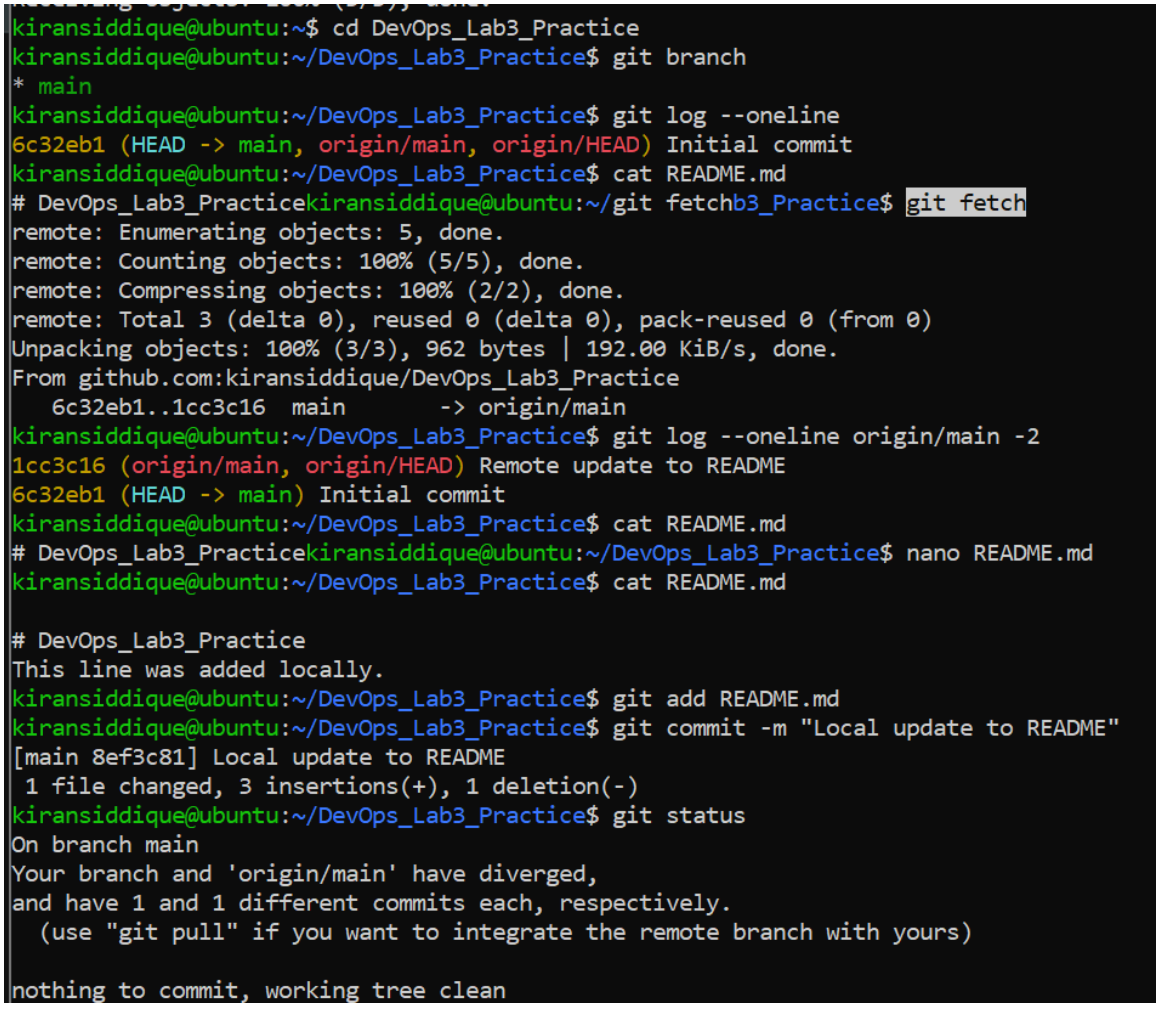


Figure 1.2: Local edit committed while the remote branch has a new commit; git status shows the branches have diverged.

Step 3: Tried to push the local commit. Git rejected the push as non-fast-forward, because the remote branch contains a commit that is not in the local branch.

```
git push origin main
```

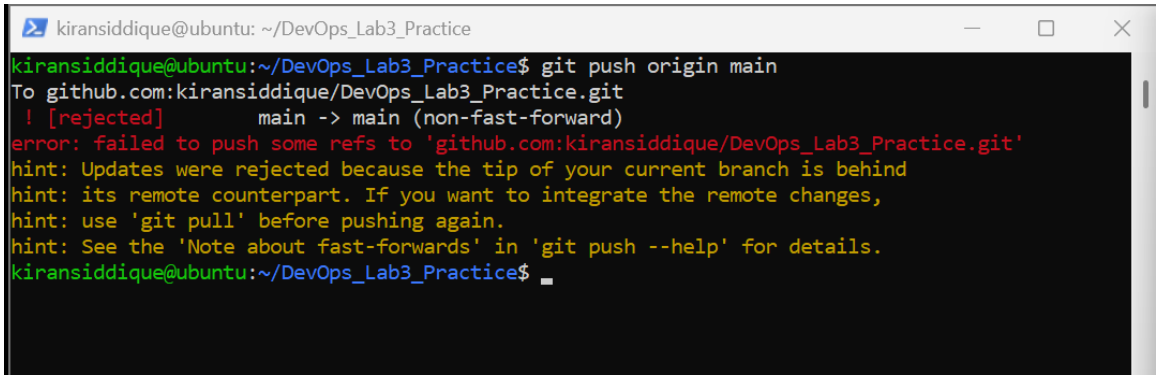


Figure 1.3: Push rejected because the remote branch has new commits.

Step 4: Pulled the remote changes with a merge (--no-rebase). Both sides changed README.md in the same area, so Git reported a merge conflict. The conflict was resolved manually by keeping both lines, removing the conflict markers, staging the file and creating the merge commit.

```
git pull --no-rebase origin main
cat README.md
nano README.md
git add README.md
git commit --no-edit
git log --oneline --graph -4
```

```
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git push origin main
To github.com:kiransiddique/DevOps_Lab3_Practice.git
 ! [rejected]        main -> main (non-fast-forward)
error: failed to push some refs to 'github.com:kiransiddique/DevOps_Lab3_Practice.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git pull --no-rebase origin main
From github.com:kiransiddique/DevOps_Lab3_Practice
 * branch            main       -> FETCH_HEAD
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ ^[[200~cat README.md~
cat: command not found
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ cat README.md
<<<<<<< HEAD

# DevOps_Lab3_Practice
This line was added locally.
=======
# DevOps_Lab3_Practice
This line was added remotely from GitHub.
>>>>>>> 1cc3c16b58c525bb12b54656bc5102cec78b8e45
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ nano README.md
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ cat README.md
# DevOps_Lab3_Practice
This line was added locally.
This line was added remotely from GitHub.
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git add README.md
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git commit --no-edit
[main e43bb54] Merge branch 'main' of github.com:kiransiddique/DevOps_Lab3_Practice
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git log --oneline --graph -4
*   e43bb54 (HEAD -> main) Merge branch 'main' of github.com:kiransiddique/DevOps_Lab3_Practice
|\
| * 1cc3c16 (origin/main, origin/HEAD) Remote update to README
* | 8ef3c81 Local update to README
|/
* 6c32eb1 Initial commit
```

Figure 1.4: Merge commit created after resolving the README.md conflict, shown in the commit graph.

Step 5: Pushed the merged history with upstream tracking. Git printed a 'This repository moved' notice because the username spelling in the remote URL differed from the GitHub account name, so the remote URL was corrected afterwards.

```
git push -u origin main
git remote set-url origin
git@github.com:KiranSiddique/DevOps_Lab3_Practice.git
```

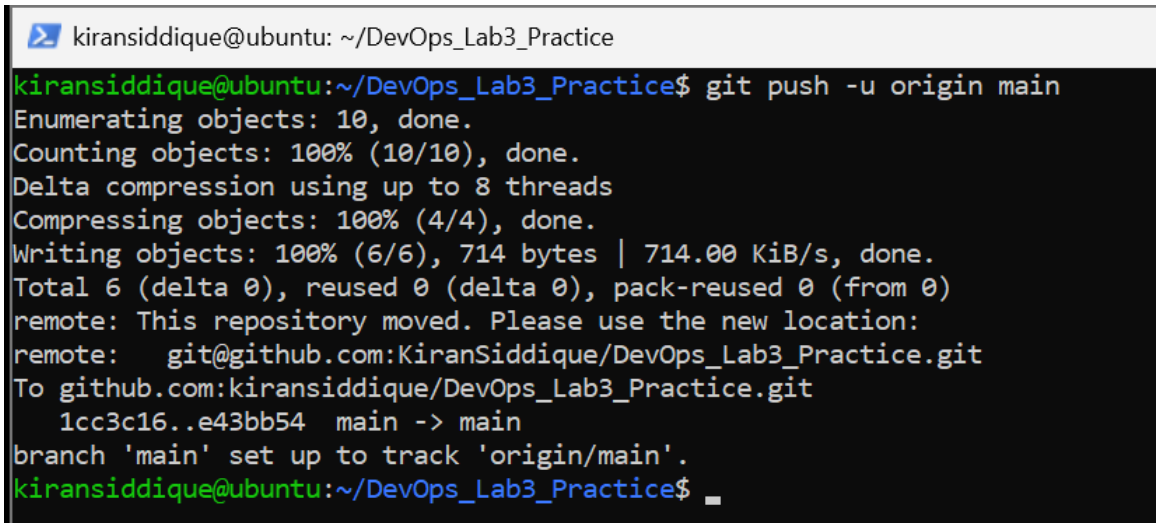


Figure 1.5: Successful push after the merge; main is set to track origin/main.

Step 6: Repeated the process with rebase. A new line was added on GitHub (This line was updated remotely again.). Locally, a new line (Second local update.) was added above the title and committed. Then git pull --rebase replayed the local commit on top of the remote commit, so the history stays linear.

```
nano README.md
git add README.md
git commit -m "Second local update"
git pull --rebase origin main
git log --oneline --graph -5
```

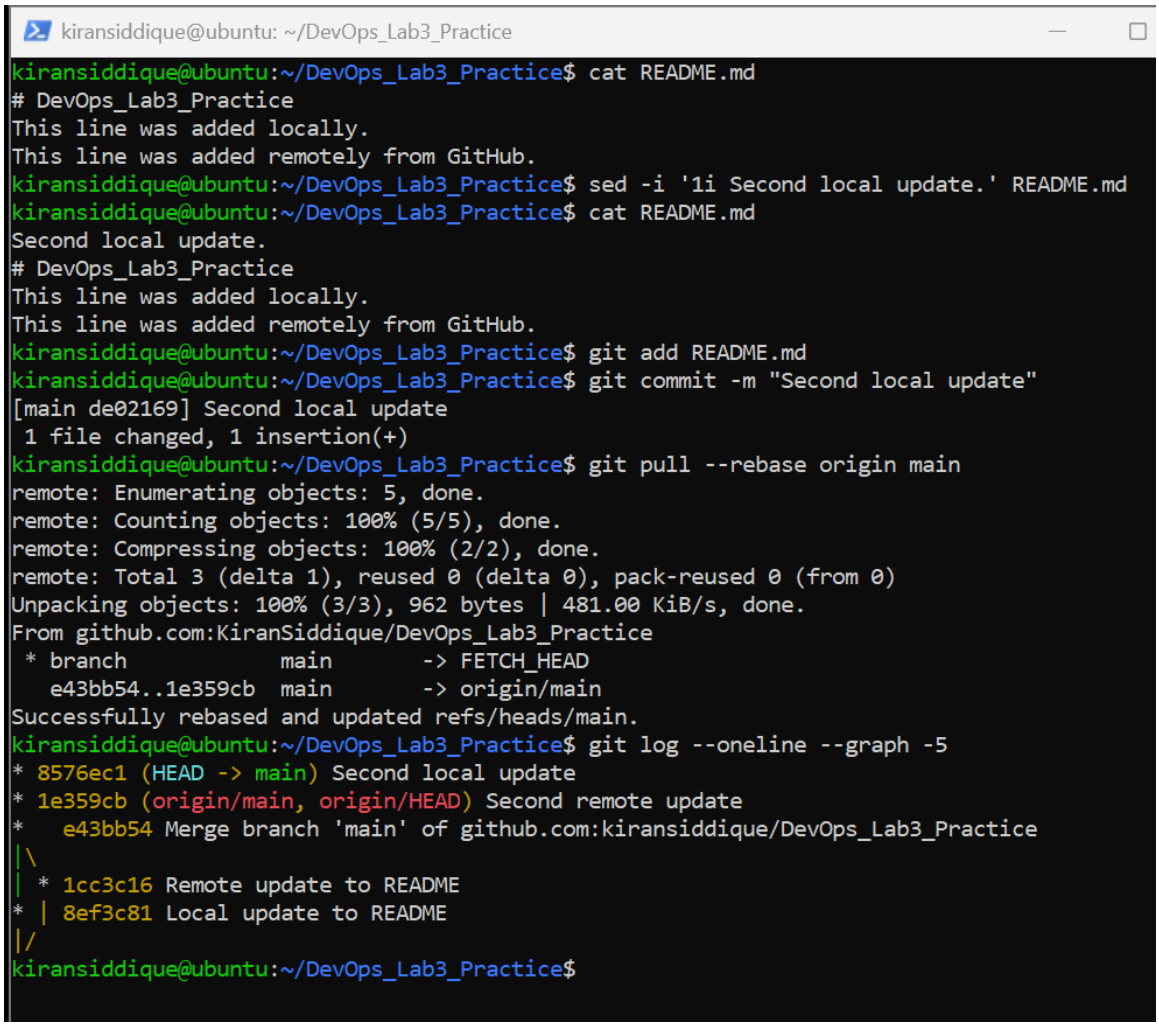


Figure 1.6: Output of git pull --rebase origin main, followed by the commit graph.

Step 7: Pushed the rebased commits to GitHub.

```
git push -u origin main
```

```
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 312 bytes | 312.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:KiranSiddique/DevOps_Lab3_Practice.git
   1e359cb..8576ec1  main -> main
branch 'main' set up to track 'origin/main'.
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$
```

Figure 1.7: Successful push after the rebase (main -> main).

Task 2: Creating and Resolving Merge Conflicts Manually

Step 1: Changed an existing line in README.md on GitHub from 'This line was updated remotely again.' to 'This line was updated remotely for conflict test.' and committed it with the message 'Remote conflicting change'.

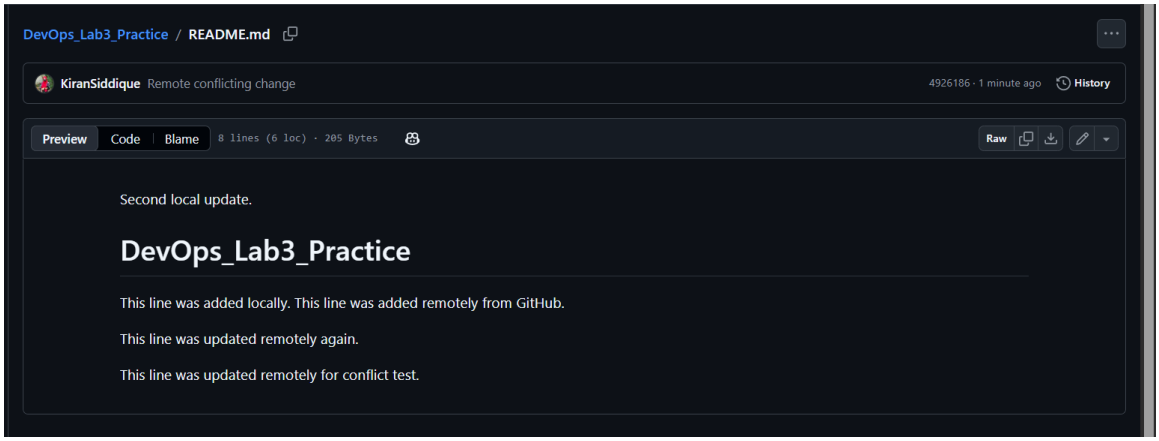


Figure 2.1: Existing line changed on GitHub and committed (Remote conflicting change).

Step 2: Changed the same line locally, without pulling first, to 'This line was updated locally at the same time.' This makes the same line different on both sides.

```
nano README.md
cat README.md
```

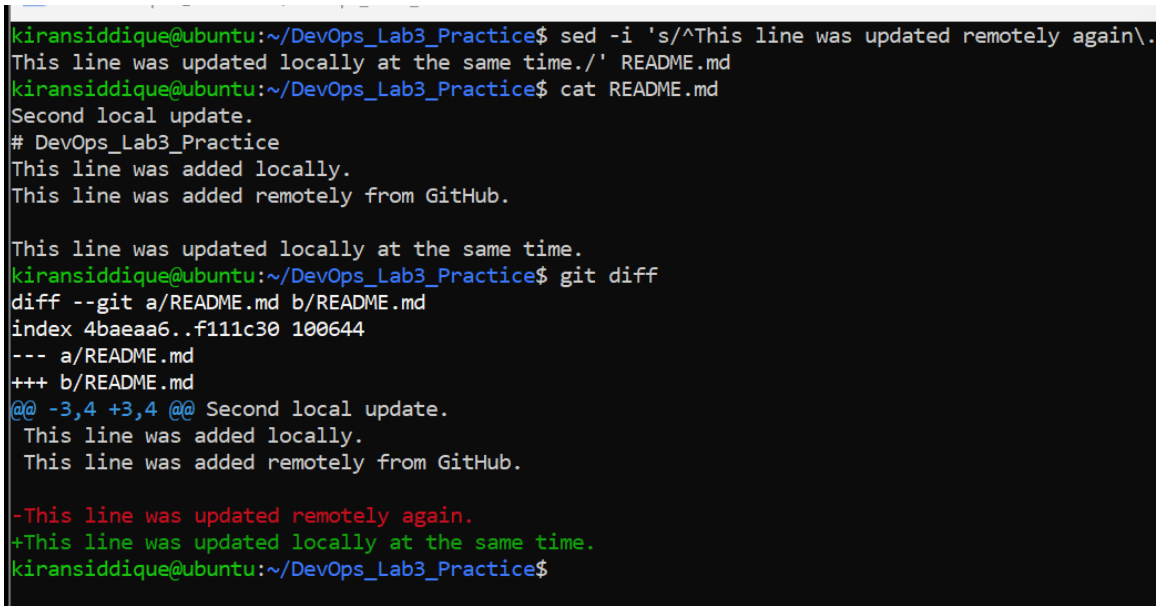


Figure 2.2: Same line changed locally in README.md before pulling.

Step 3: Staged and committed the local change with the message 'Local conflicting change'.

```
git add README.md
git commit -m "Local conflicting change"
```

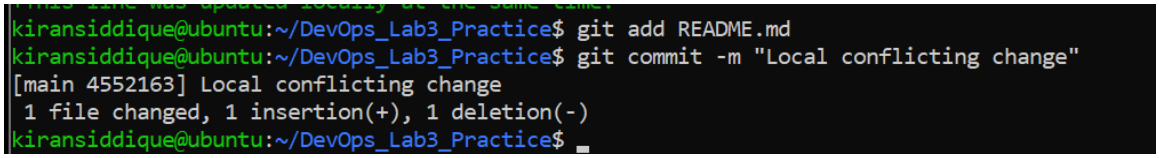


Figure 2.3: Local conflicting change committed.

Step 4: Tried to push. The push was rejected because the remote branch has a commit that conflicts with the local commit.

```
git push -u origin main
```

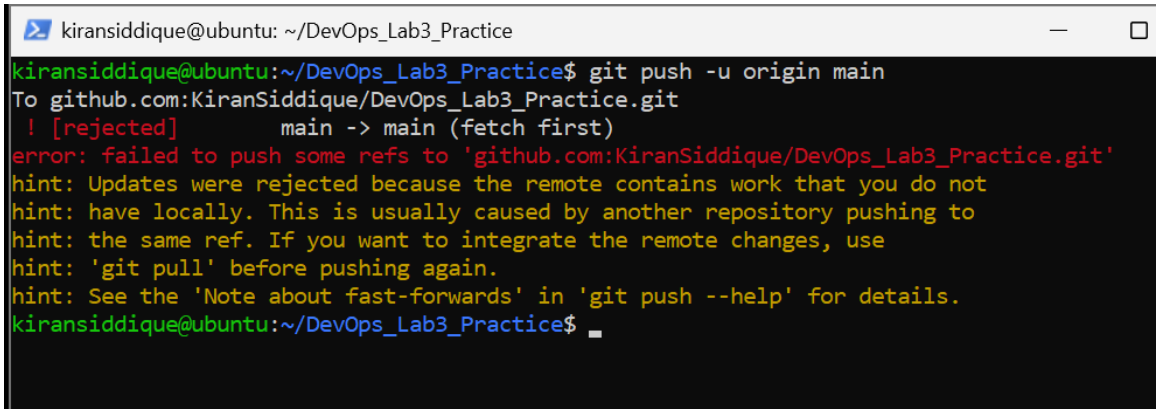


Figure 2.4: Push rejected because the remote has conflicting changes.

Step 5: Pulled with rebase. Git tried to replay the local commit on top of the remote commit, found the same line changed on both sides, and stopped with a merge conflict in README.md.

```
git pull --rebase origin main
```

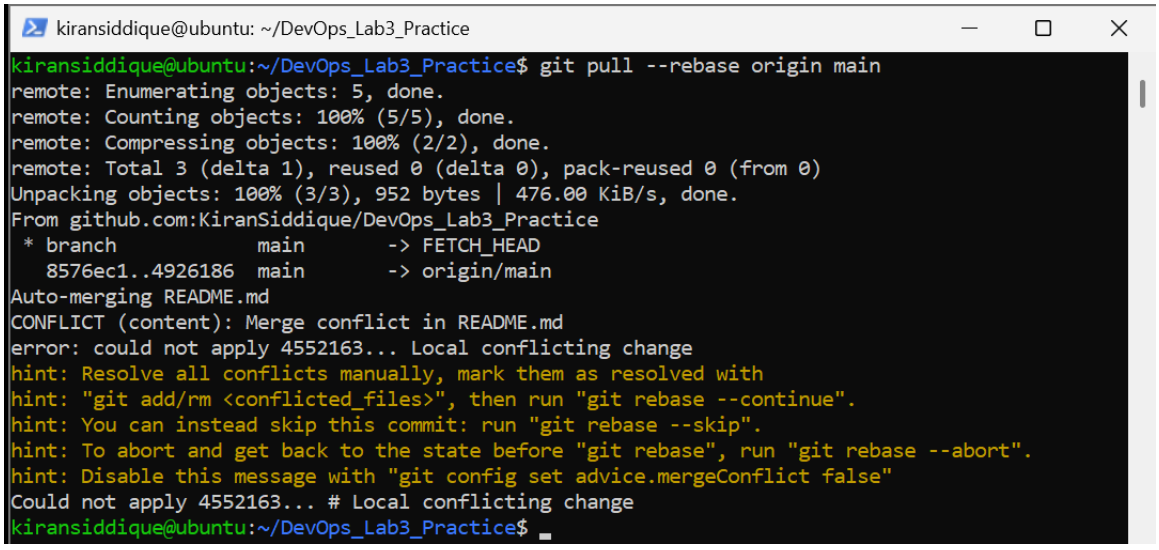


Figure 2.5: git pull --rebase reports a merge conflict in README.md.

Step 6: Opened README.md, removed the conflict markers (<<<<<<< HEAD, =======, and >>>>>>>) and kept both lines. The file was then checked with cat, and it contains both lines with no markers.

```
nano README.md
cat README.md
```

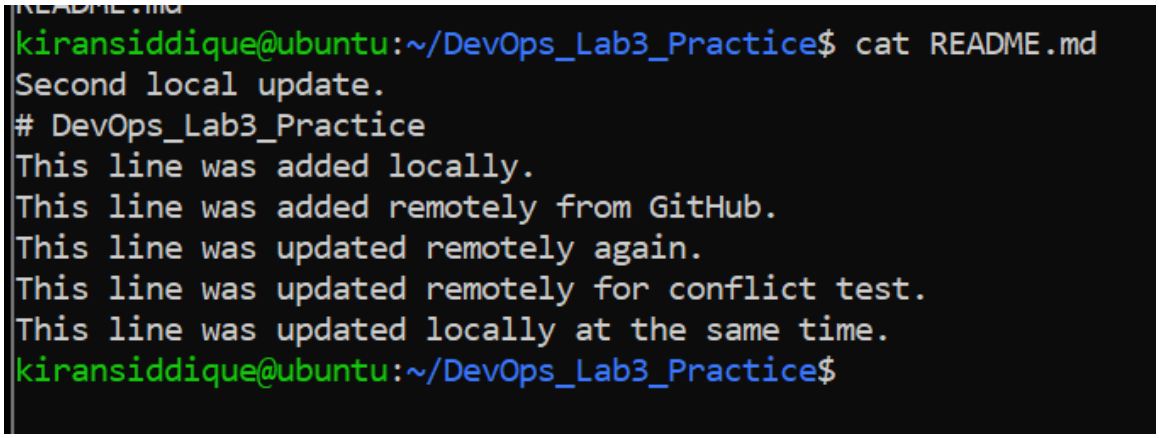


Figure 2.6: README.md after the conflict markers were removed and both lines were kept.

Step 7: Marked the conflict as resolved by staging the file, then continued the rebase.

```
git add README.md
git rebase --continue
```

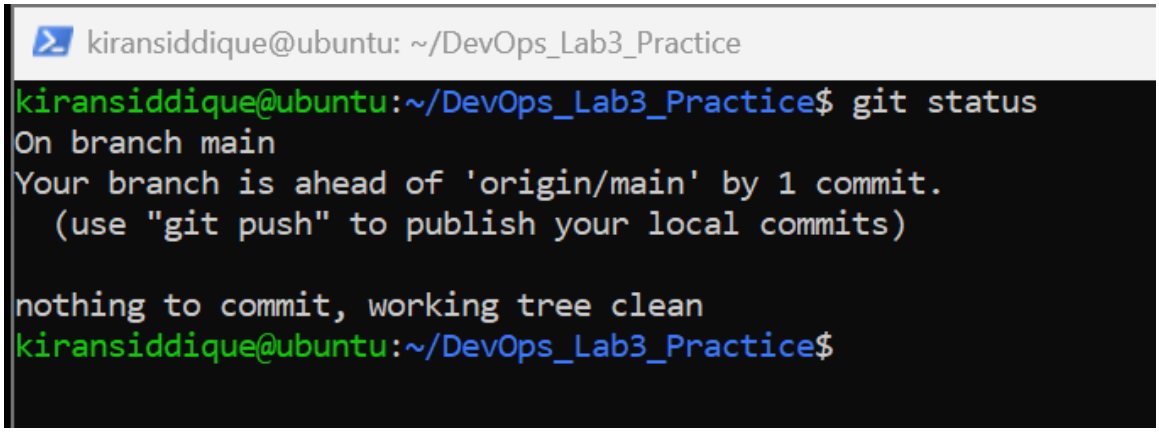


Figure 2.7: Conflict marked as resolved with git add, and the rebase continued.

Step 8: Pushed the resolved commit to GitHub.

```
git push -u origin main
```

```
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 360 bytes | 360.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:KiranSiddique/DevOps_Lab3_Practice.git
   4926186..1907d5e  main -> main
branch 'main' set up to track 'origin/main'.
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$
```

Figure 2.8: Successful push of the resolved commit (main -> main).

Task 3: Managing Ignored Files with .gitignore and Removing Tracked Files

Step 1: Created a textfiles directory with three files (a.txt, b.txt and c.txt), committed them and pushed them to GitHub.

```
mkdir textfiles
echo "File A content" > textfiles/a.txt
echo "File B content" > textfiles/b.txt
echo "File C content" > textfiles/c.txt
git add .
git commit -m "Added textfiles directory with three files"
git push origin main
```

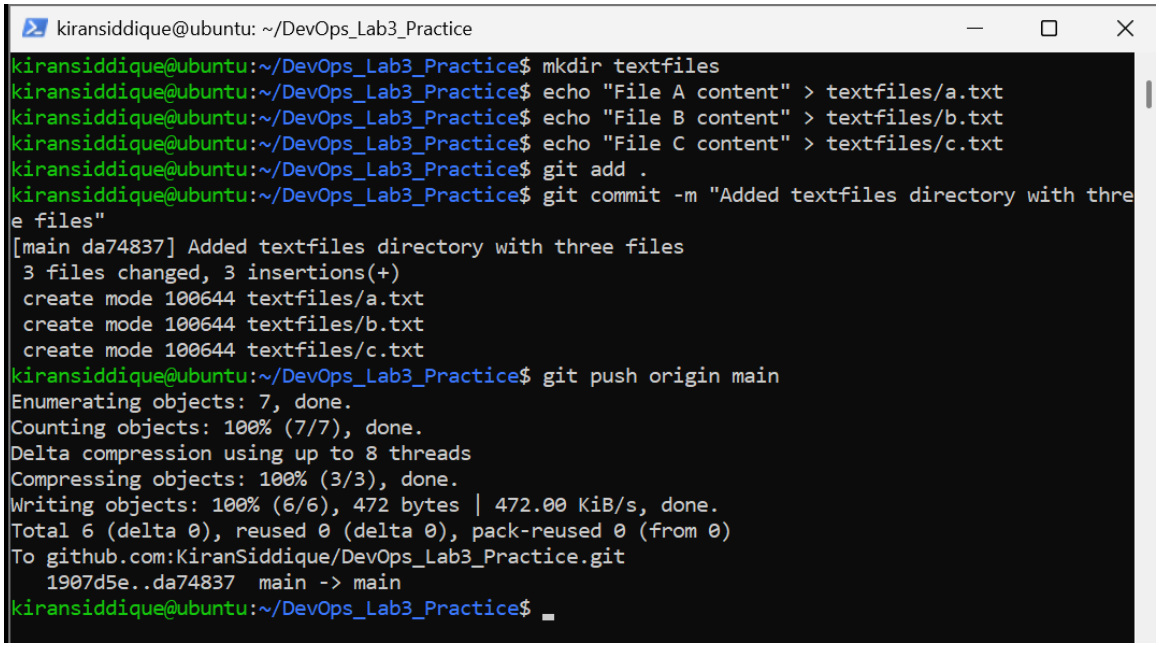


Figure 3.1: textfiles directory with three files committed and pushed.

Step 2: Created a .gitignore file containing textfiles/*, which tells Git to ignore the directory. The file was committed and pushed.

```
echo "textfiles/*" > .gitignore
git add .gitignore
git commit -m "Added .gitignore to ignore textfiles directory"
git push origin main
```

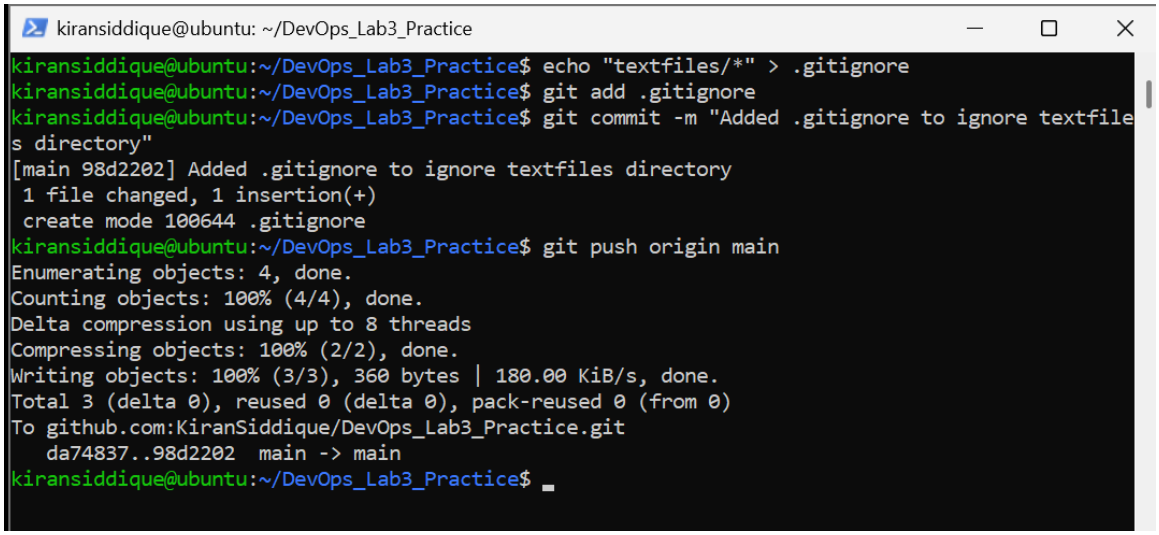


Figure 3.2: .gitignore committed and pushed.

Step 3: Checked the repository on GitHub. The textfiles directory is still visible, because .gitignore does not remove files that Git is already tracking.

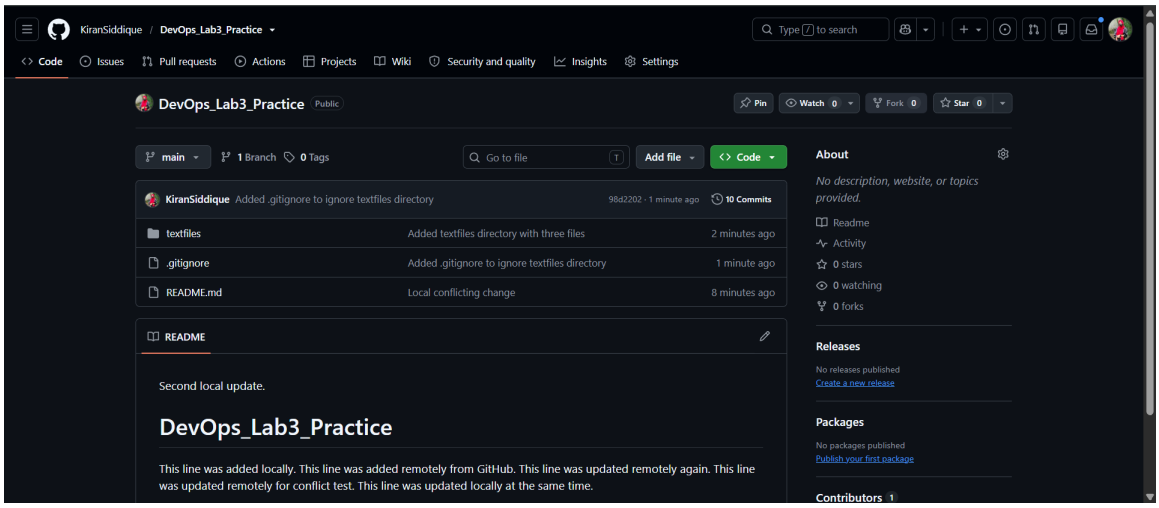


Figure 3.3: textfiles directory is still visible on GitHub because it was already tracked.

Step 4: Removed the directory from tracking with git rm -r --cached. The files stay on the local disk. The change was committed and pushed, and ls confirms that the local files still exist.

```
git rm -r --cached textfiles
git add .
git commit -m "Removed tracked textfiles directory"
git push origin main
ls textfiles
```

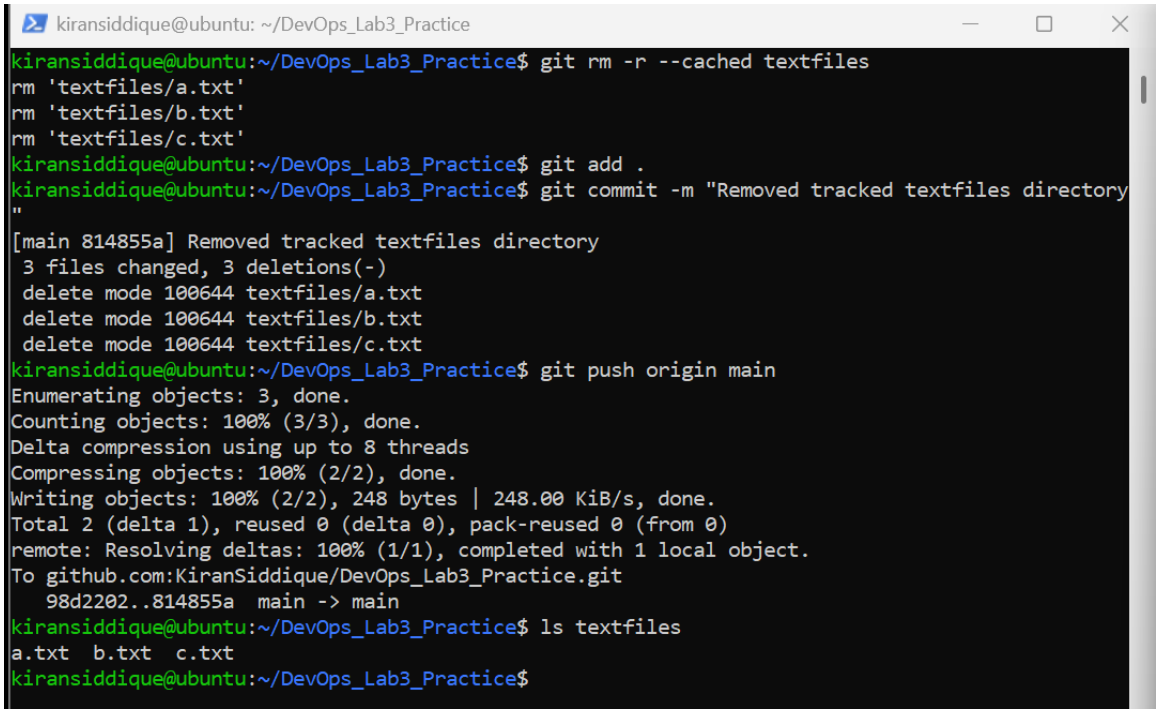


Figure 3.4: Directory removed from tracking and pushed; the local files are still listed by ls textfiles.

Step 5: Refreshed the GitHub repository page. The textfiles directory is no longer shown.

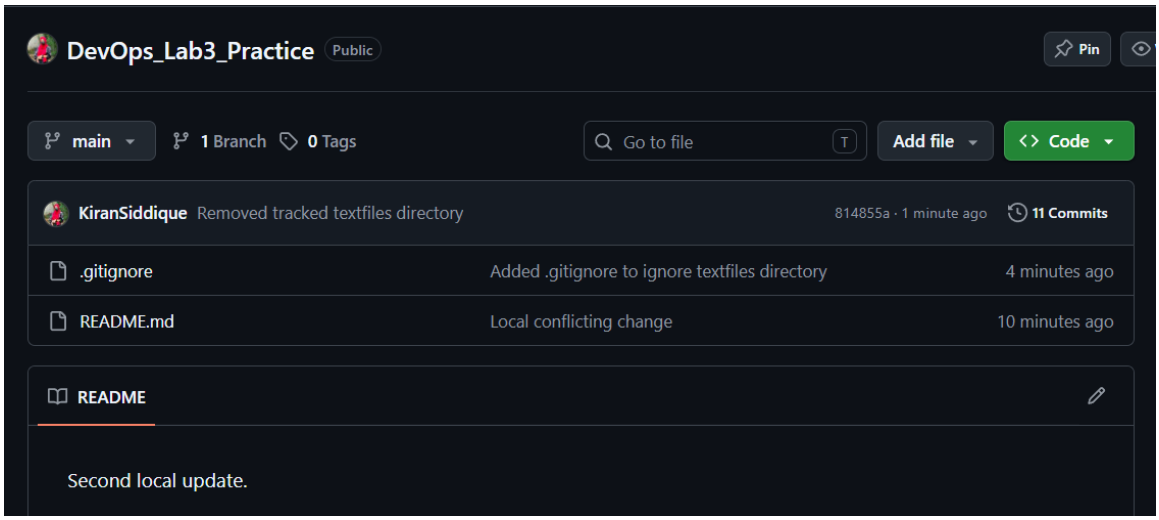


Figure 3.5: textfiles directory removed from the GitHub repository.

Task 4: Create Temporary Changes and Use git stash

Step 1: Created feature-branch, added two lines to README.md, and committed the change.

```
git checkout -b feature-branch
echo "Feature branch line 1" >> README.md
echo "Feature branch line 2" >> README.md
git add README.md
git commit -m "Changes the README.md file"
```

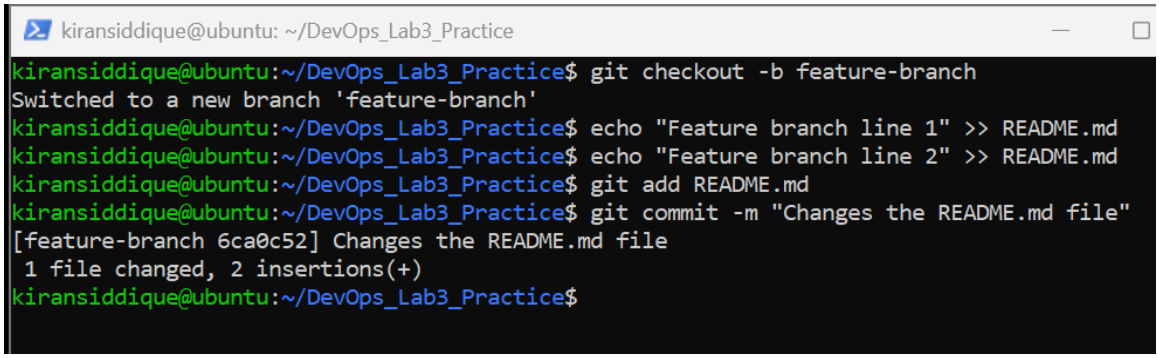


Figure 4.1: feature-branch created and the README.md change committed.

Step 2: Added two uncommitted lines to README.md and tried to switch to main. Git refused, because the local changes would be overwritten by the checkout.

```
echo "Uncommitted change line 1" >> README.md
echo "Uncommitted change line 2" >> README.md
git checkout main
```

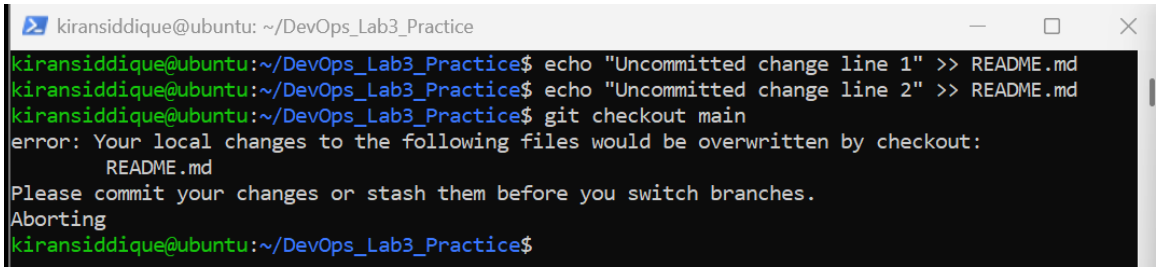


Figure 4.2: git checkout main refused because of uncommitted changes.

Step 3: Saved the uncommitted changes with git stash.

```
git stash
```

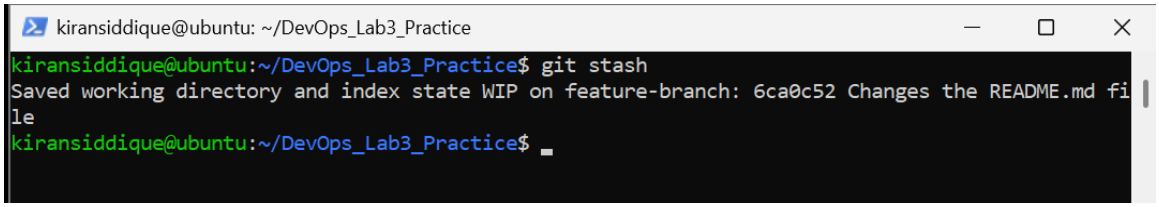


Figure 4.3: Uncommitted changes saved with git stash.

Step 4: Switched to main. This works now, because the working tree is clean.

```
git checkout main
```

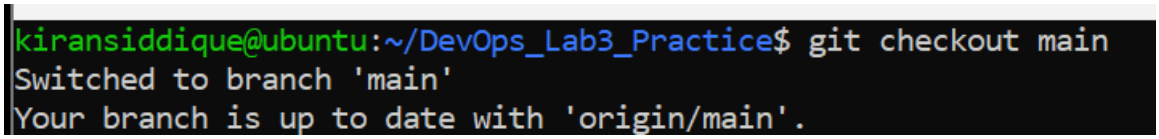


Figure 4.4: Switched to main after stashing the changes.

Step 5: Switched back to feature-branch.

```
git checkout feature-branch
```

```
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git checkout feature-branch
Switched to branch 'feature-branch'
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$
```

Figure 4.5: Switched back to feature-branch.

Step 6: Checked the working tree status, which is clean.

```
git status
```

```
kiransiddique@ubuntu: ~/DevOps_Lab3_Practice
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git status
On branch feature-branch
nothing to commit, working tree clean
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$
```

Figure 4.6: git status shows a clean working tree.

Step 7: Restored the stashed changes with git stash pop. The stash entry is removed after it is applied.

```
git stash pop
```

```
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git stash pop
On branch feature-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (e5f927d0950c73806ce672c1991d2c4714597f85)
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$
```

Figure 4.7: Stashed changes restored with git stash pop.

Task 5: Checkout a Specific Commit Using git log

Step 1: Viewed the commit history with git log --oneline.

```
git log --oneline
```

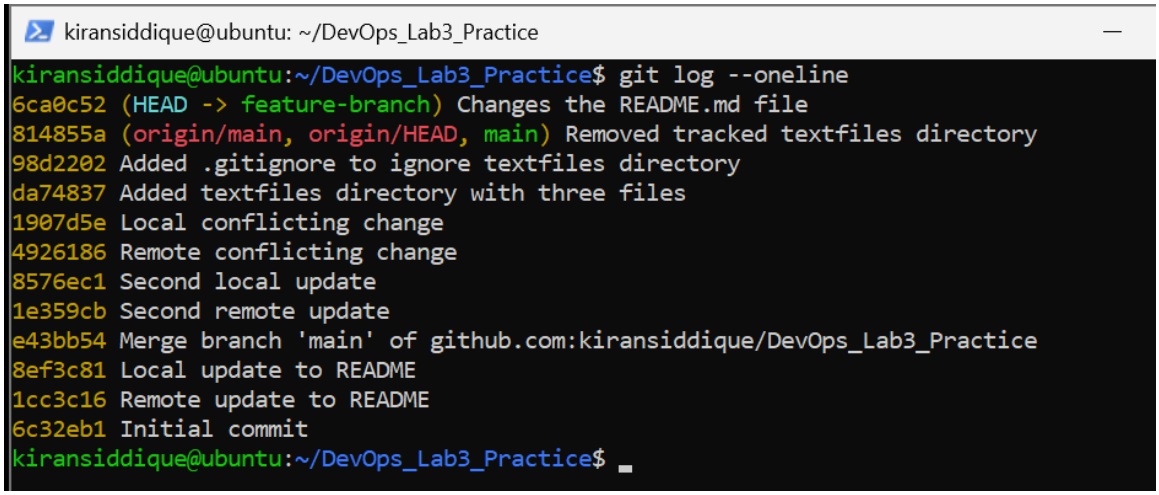


Figure 5.1: Commit history shown with git log --oneline.

Step 2: Checked out an older commit (6c32eb1, Initial commit) by its hash. Because the uncommitted change would block the checkout, it was stashed first. Git reports the detached HEAD state, where commits are not attached to a branch.

```
git stash
git checkout 6c32eb1
```

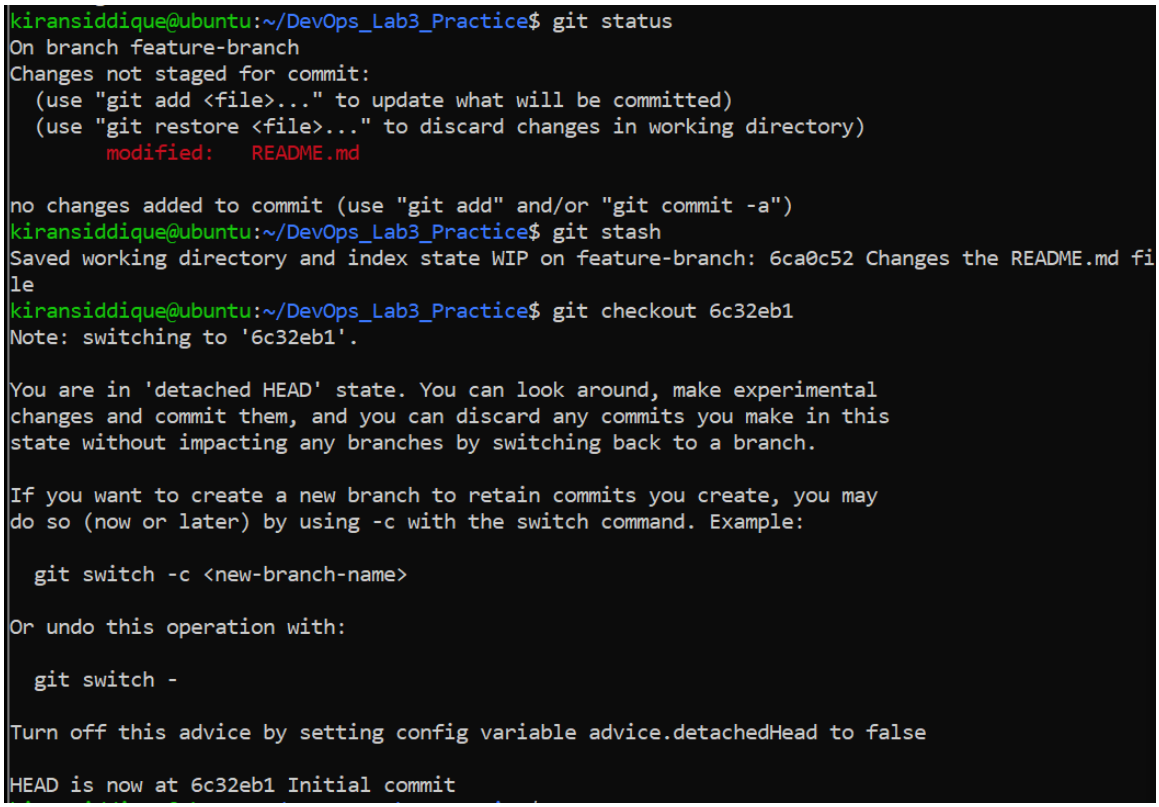


Figure 5.2: Detached HEAD state after checking out an older commit.

Step 3: Returned to the main branch.

```
git checkout main
```

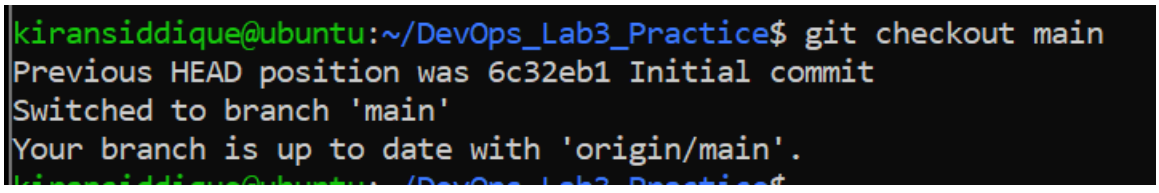


Figure 5.3: Returned to the main branch.

Task 6: Resetting Commits (Soft vs Hard Reset)

Step 1: Added a line 'Test line 1' to README.md and committed it with the message 'Added test line'.

```
echo "Test line 1" >> README.md
git add .
git commit -m "Added test line"
```

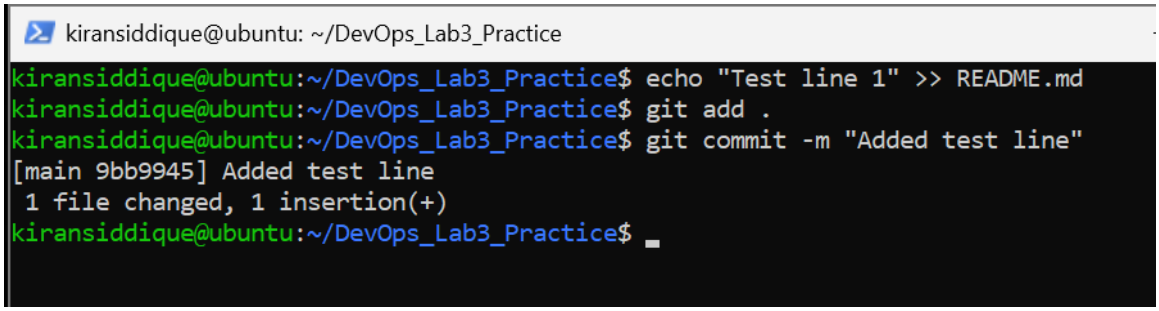


Figure 6.1: First test change committed (Added test line).

Step 2: Added a second line 'Test line 2' and committed it with the message 'Second test commit'.

```
echo "Test line 2" >> README.md
git add .
git commit -m "Second test commit"
```

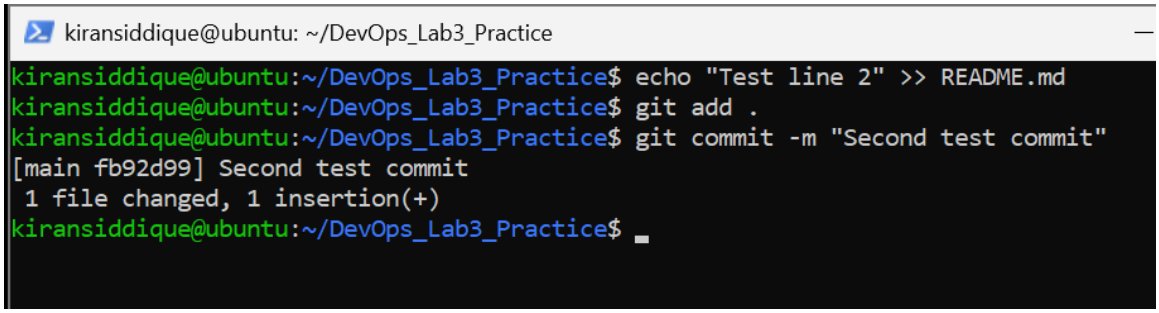


Figure 6.2: Second test change committed (Second test commit).

Step 3: Viewed the commit history and the file before the reset. README.md contains both test lines.

```
git log --oneline
cat README.md
```

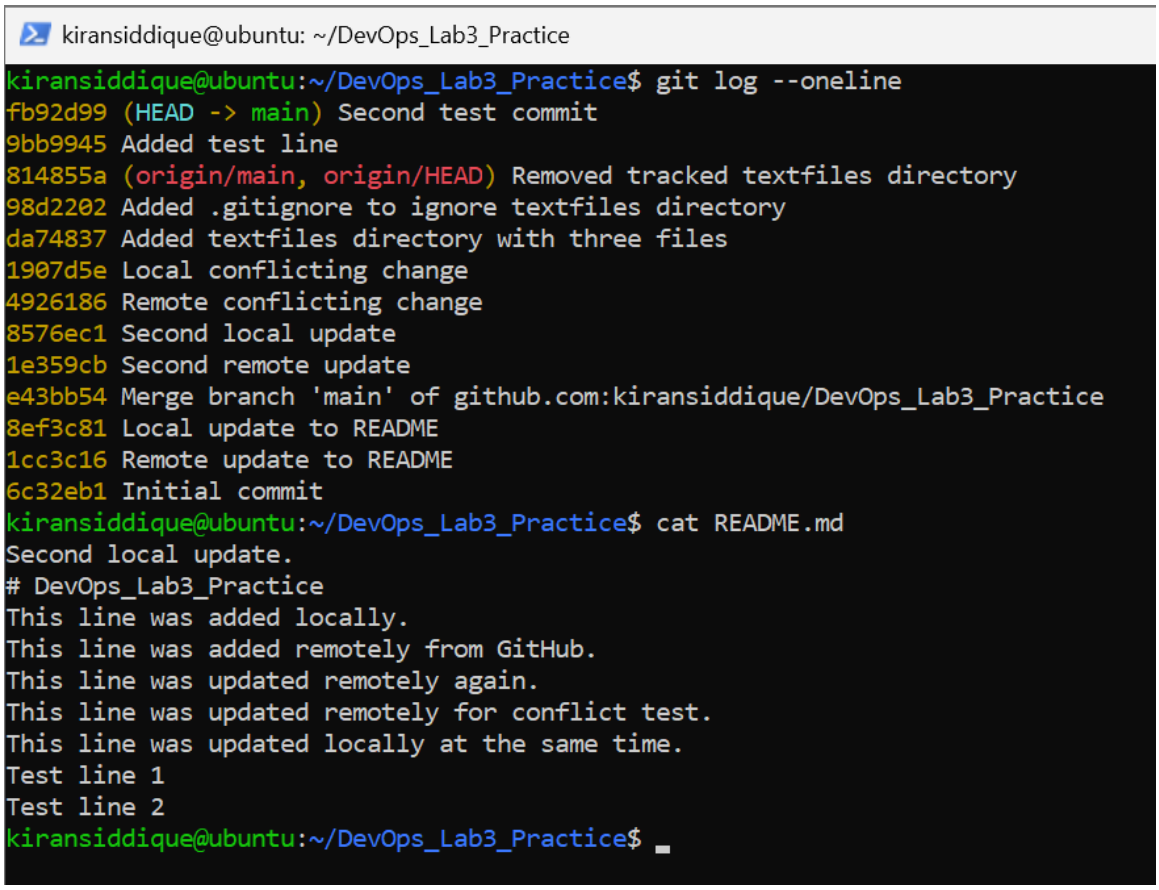


Figure 6.3: Commit history before the reset.

Step 4: Checked the README.md content before the reset.

```
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ cat README.md
Second local update.
# DevOps_Lab3_Practice
This line was added locally.
This line was added remotely from GitHub.
This line was updated remotely again.
This line was updated remotely for conflict test.
This line was updated locally at the same time.
Test line 1
Test line 2
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$
```

Figure 6.4: README.md contains both test lines before the reset.

Step 5: Performed a soft reset with git reset --soft HEAD~1. The last commit is removed, but its changes stay staged.

```
git reset --soft HEAD~1
git status
```

```
kiransiddique@ubuntu: ~/DevOps_Lab3_Practice
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git reset --soft HEAD~1
kiransiddique@ubuntu:~/DevOps_Lab3_Practice$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

kiransiddique@ubuntu:~/DevOps_Lab3_Practice$
```

Figure 6.5: git reset --soft HEAD~1 keeps the changes staged.

Step 6: Checked the commit history after the soft reset. The last commit is no longer in the history.

```
git log --oneline
cat README.md
```

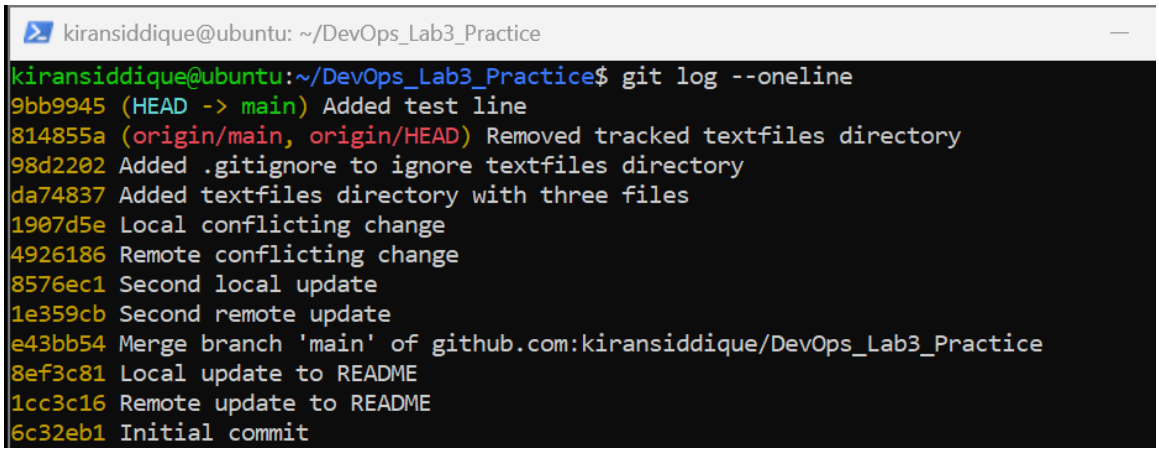


Figure 6.6: Commit history after the soft reset.

Step 7: Checked README.md and the working tree after the soft reset. Both test lines are still present in the file, and the changes are staged.

```
cat README.md
git status
```

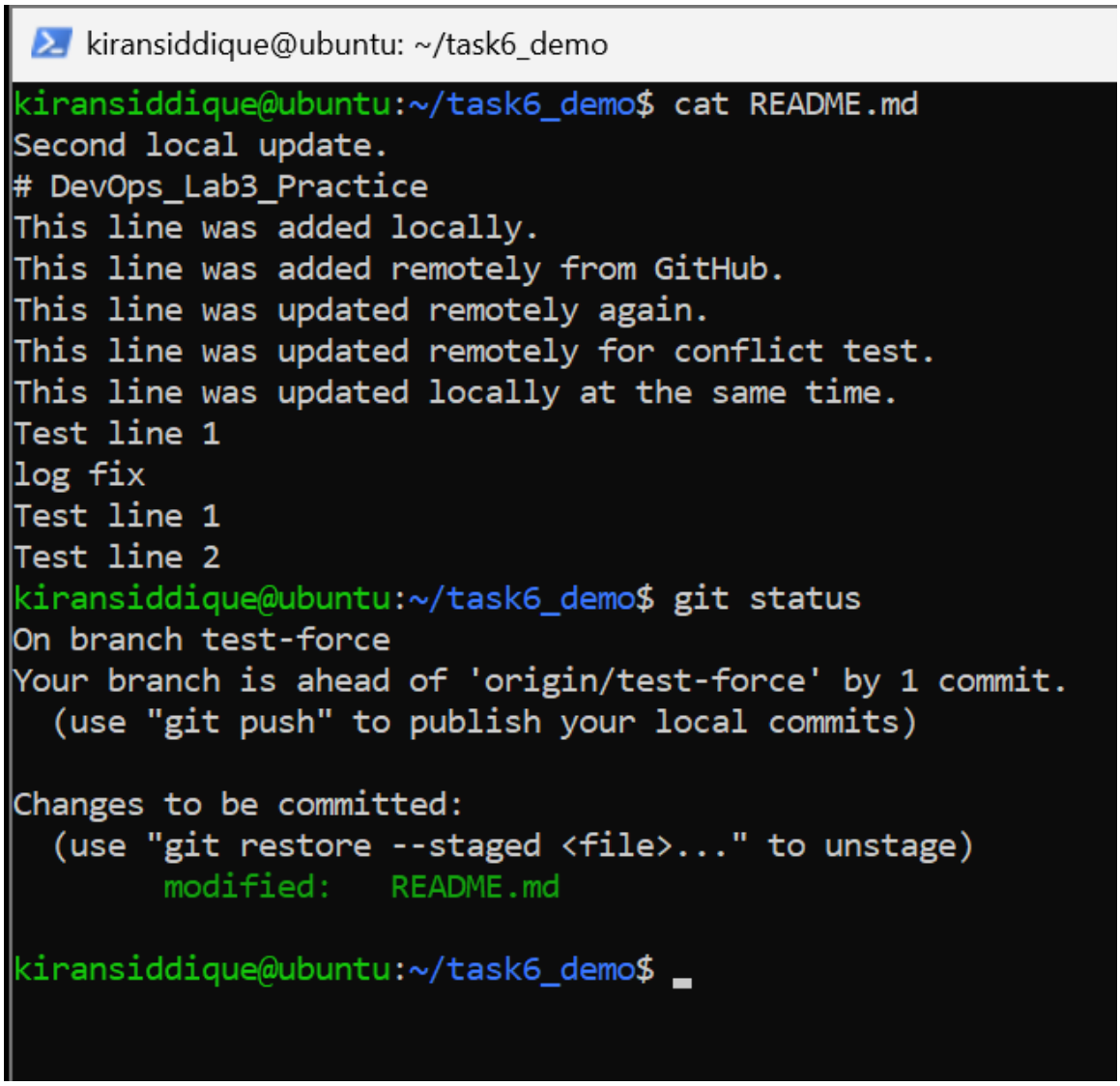


Figure 6.7: README.md and git status after the soft reset; both edits are still present and staged.

Step 8: Committed the staged changes again with the message 'Second Test commit'.

```
git commit -m "Second Test commit"
```

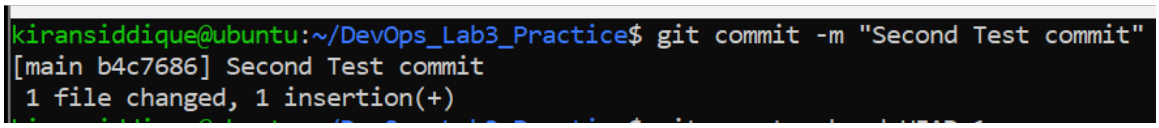


Figure 6.8: Staged changes committed again (Second Test commit).

Step 9: Performed a hard reset with git reset --hard HEAD~1. The last commit and its changes are discarded.

```
git reset --hard HEAD~1
```

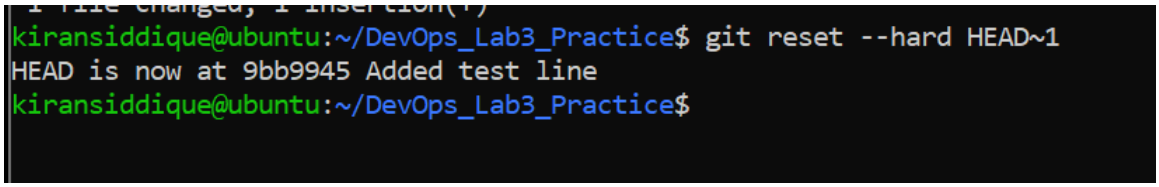


Figure 6.9: git reset --hard HEAD~1 discards the last commit and its changes.

Step 10: Checked the commit history after the hard reset.

```
git log --oneline
```

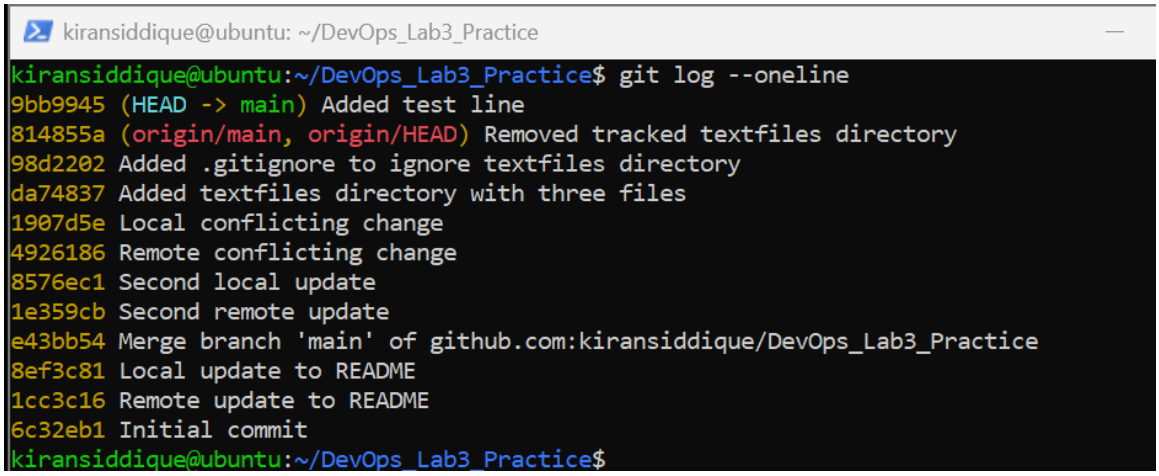


Figure 6.10: Commit history after the hard reset.

Step 11: Checked README.md and the working tree status. The second edit is gone, and the working tree is clean.

```
cat README.md
git status
```

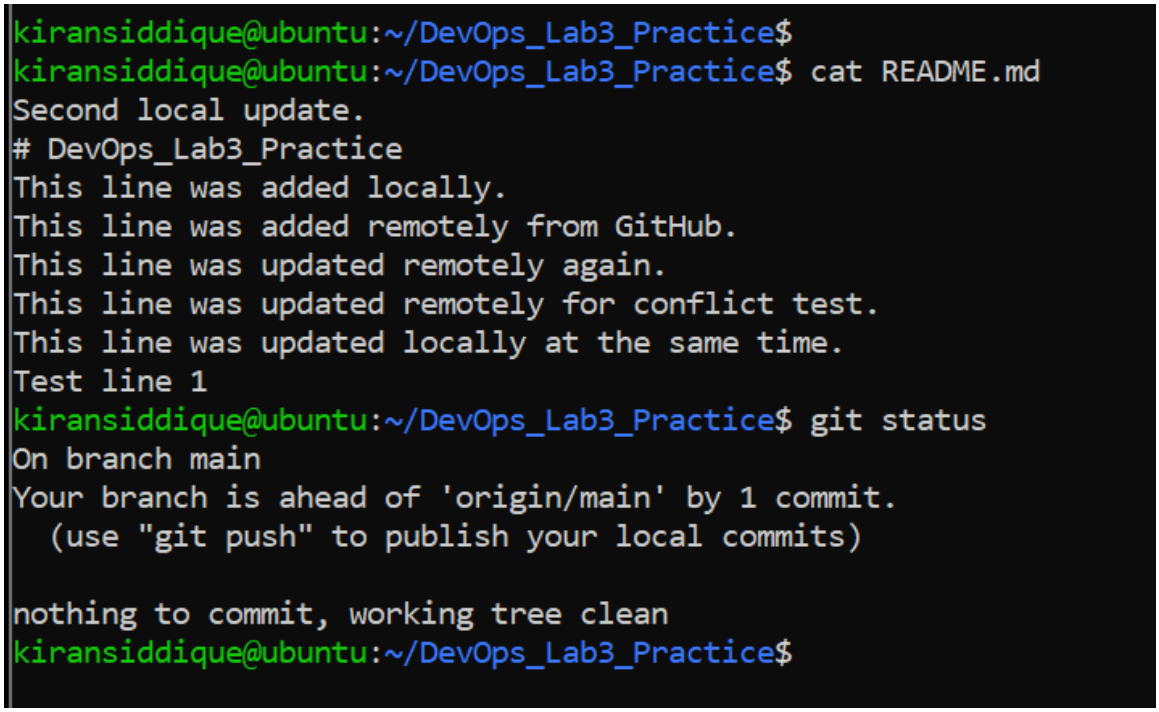


Figure 6.11: README.md and git status after the hard reset.

Task 7: Amending the Last Commit

Step 1: Made a change to README.md and committed it with the message 'Fix log message'.

```
echo "log fix" >> README.md
git add .
git commit -m "Fix log message"
```

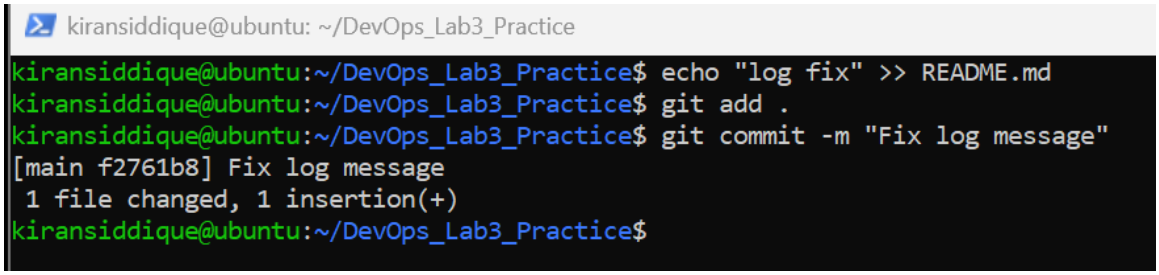


Figure 7.1: Change committed with the message Fix log message.

Step 2: Changed another file, notes.txt, and amended the last commit so that both files are in one commit. git show --stat HEAD lists both changed files.

```
echo "notes line" > notes.txt
git add .
git commit --amend --no-edit
git show --stat HEAD
```

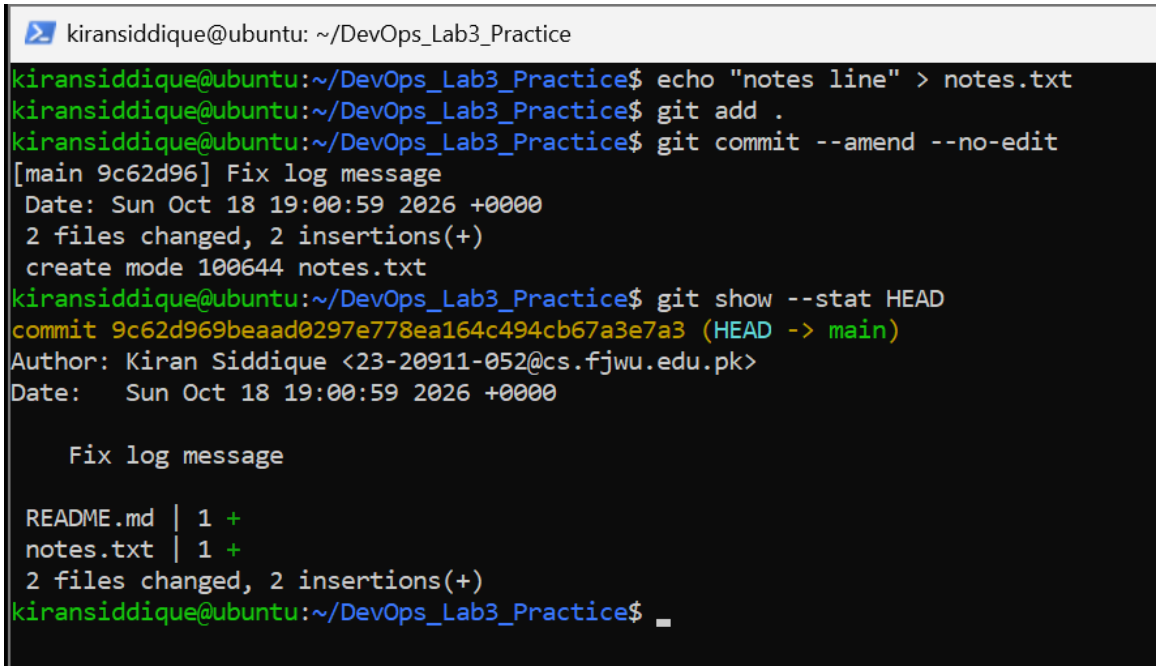


Figure 7.2: Last commit amended with notes.txt; git show --stat HEAD lists both files.

Task 8: Reverting a Commit (Safe Undo on Remote Branch)

Step 1: Created temp.txt, committed it with the message 'Added temporary text file' and pushed it to main.

```
echo "temporary text" >> temp.txt
git add .
git commit -m "Added temporary text file"
git push origin main
```

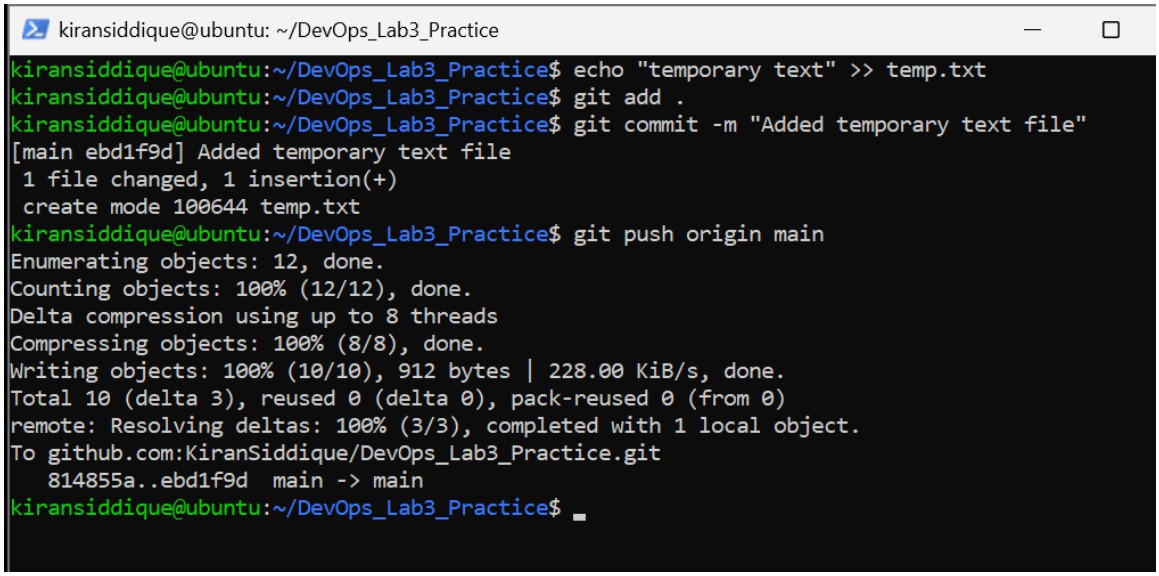


Figure 8.1: temp.txt committed and pushed to main.

Step 2: Reverted the commit with git revert. A new commit is created that undoes the change, and the history is not rewritten.

```
git log --oneline -3
git revert --no-edit <hash>
git log --oneline -4
```

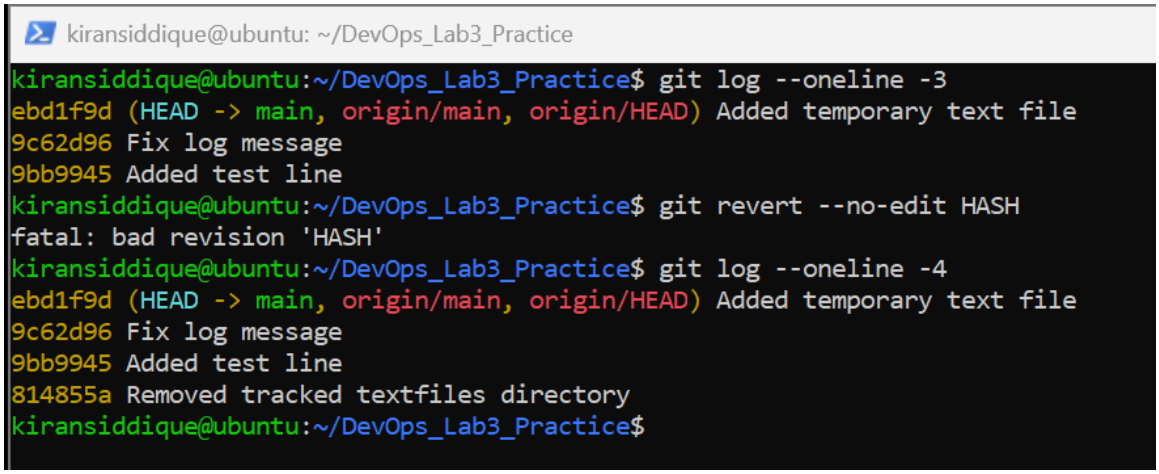


Figure 8.2: git revert creates a new commit that undoes Added temporary text file.

Step 3: Pushed the revert commit to GitHub.

```
git push origin main
```

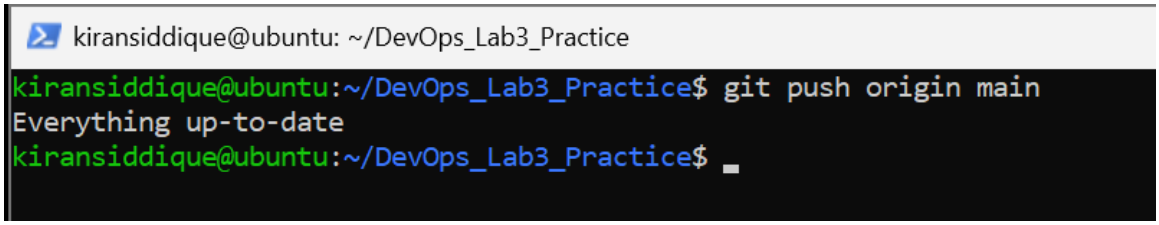


Figure 8.3: Revert commit pushed to GitHub.

Task 9: Force Push (With Caution)

Step 1: Created a new branch test-force. This was done only on a test branch, never on main.

```
git checkout -b test-force
```

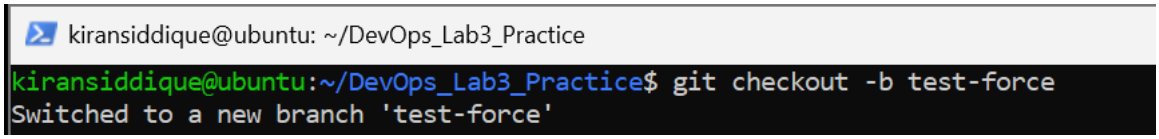


Figure 9.1: test-force branch created.

Step 2: Added a small change in force.txt and committed it on test-force.

```
echo "force test line" >> force.txt
git add .
git commit -m "Force test commit"
```

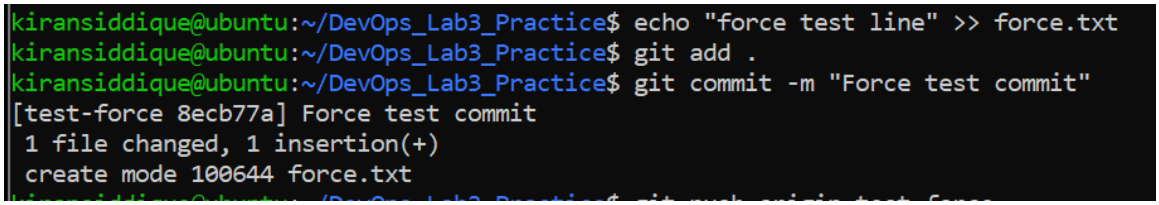


Figure 9.2: Small change committed on test-force.

Step 3: Pushed test-force to GitHub.

```
git push origin test-force
```

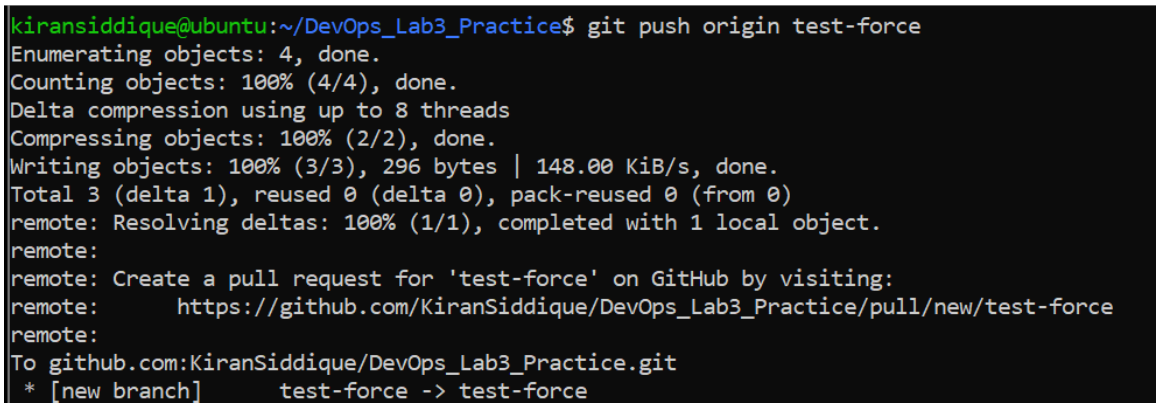


Figure 9.3: test-force pushed to GitHub.

Step 4: Removed the last commit locally with git reset --hard HEAD~1.

```
git reset --hard HEAD~1
```

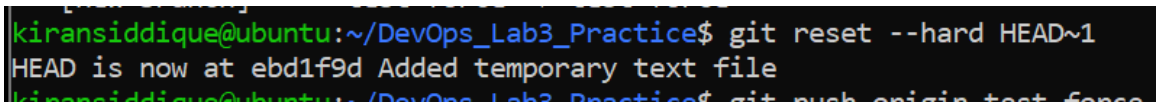


Figure 9.4: Local test-force reset one commit back.

Step 5: Tried a normal push. The remote rejected it as non-fast-forward, because the remote branch still has the commit that was removed locally.

```
git push origin test-force
```

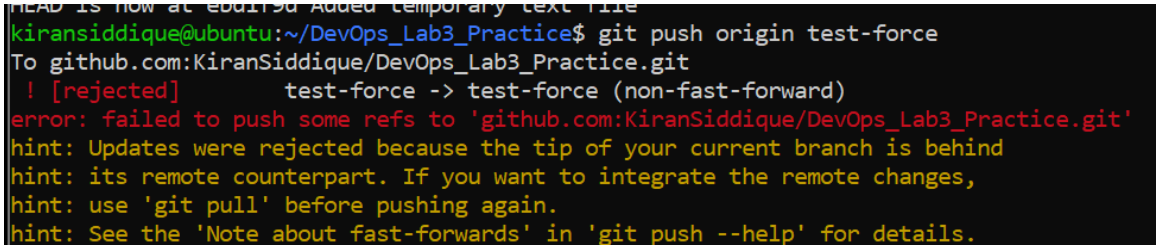


Figure 9.5: Normal push rejected by the remote (non-fast-forward).

Step 6: Force-pushed the branch. The remote branch history was rewritten to match the local branch.

```
git push origin test-force --force
```

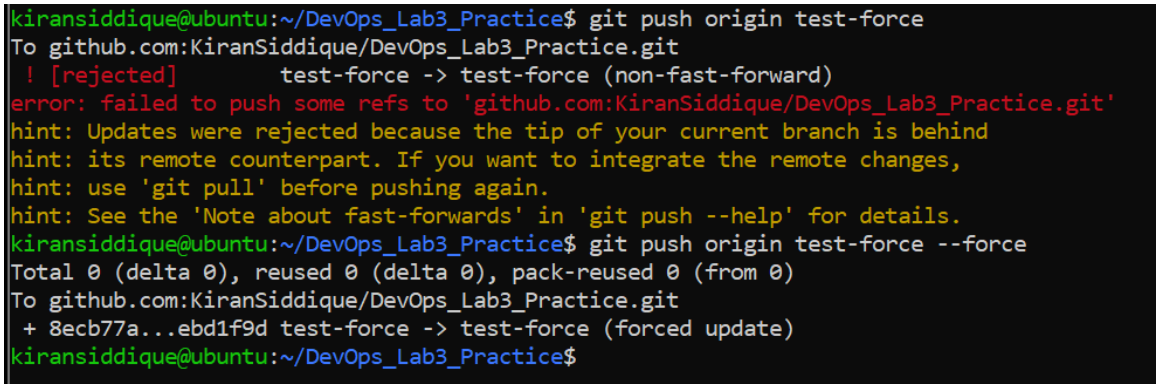


Figure 9.6: Forced update of test-force on the remote.

Task 10: Running Gitea in GitHub Codespaces via Docker Compose

Step 1: Forked the Gitea repository (WaqasSaleem97/Gitea, provided for this lab) into my GitHub account.

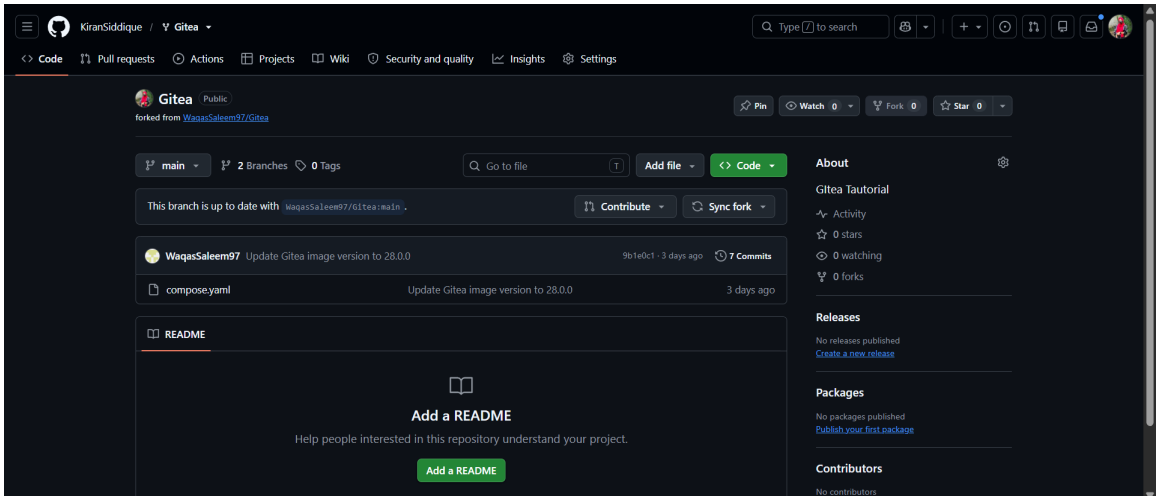


Figure 10.1: Gitea repository forked into my GitHub account.

Step 2: Opened the Codespaces panel of the fork. A codespace was already active on the main branch.

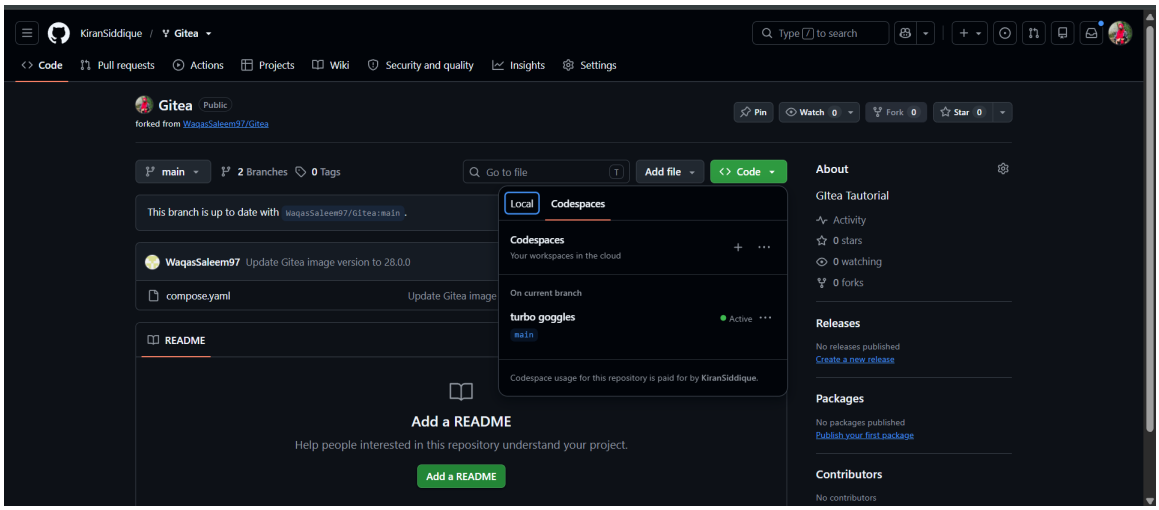


Figure 10.2: Codespaces panel showing the codespace created from the fork.

Step 3: Created a Codespace and waited while it loaded.

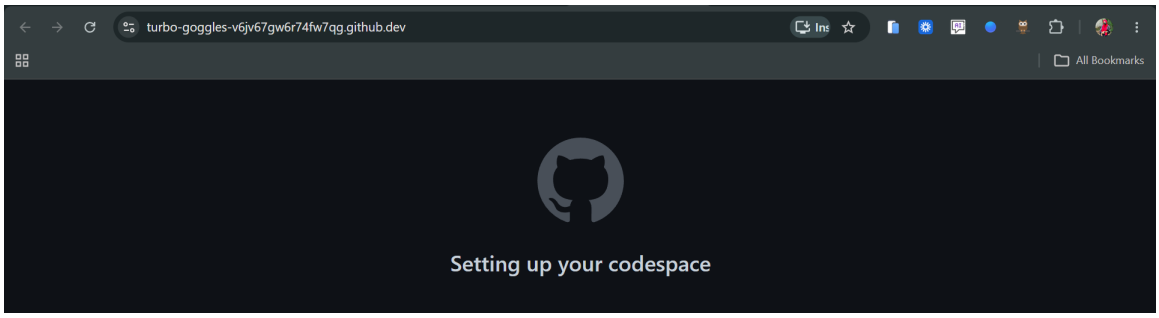


Figure 10.3: Codespace loading (Setting up your codespace).

Step 4: In the Codespace terminal, switched to the codespace branch, restarted the containers from that branch's compose file, and checked that the gitea and gitea_db containers are running.

```
git checkout codespace
docker compose down
docker compose up -d
docker compose ps
```

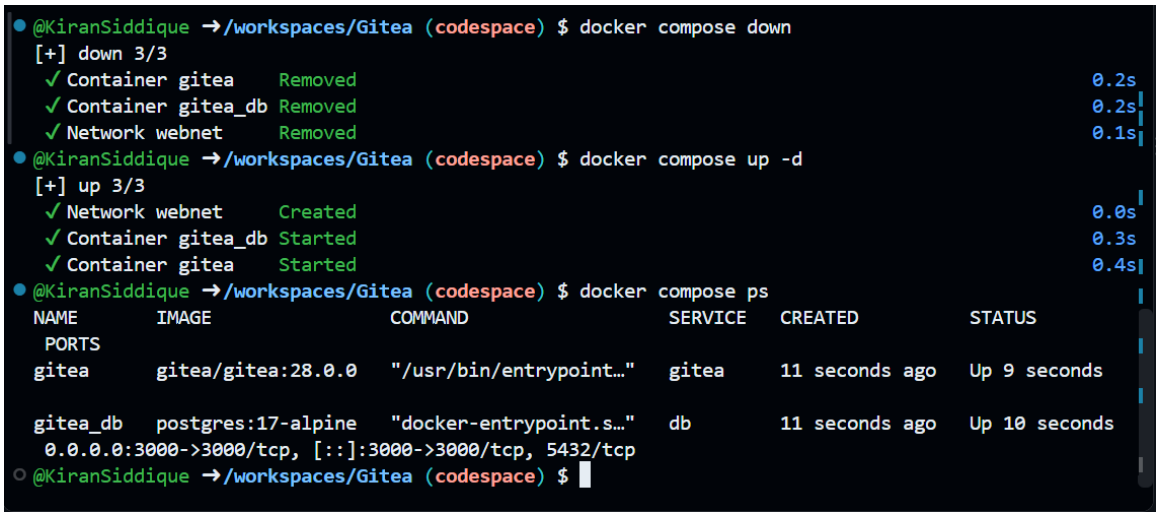


Figure 10.4: docker compose ps output with the Gitea containers running.

Step 5: Opened the PORTS tab, added port 3000, and opened the Port Visibility options from its right-click menu. At this point the port is Private.

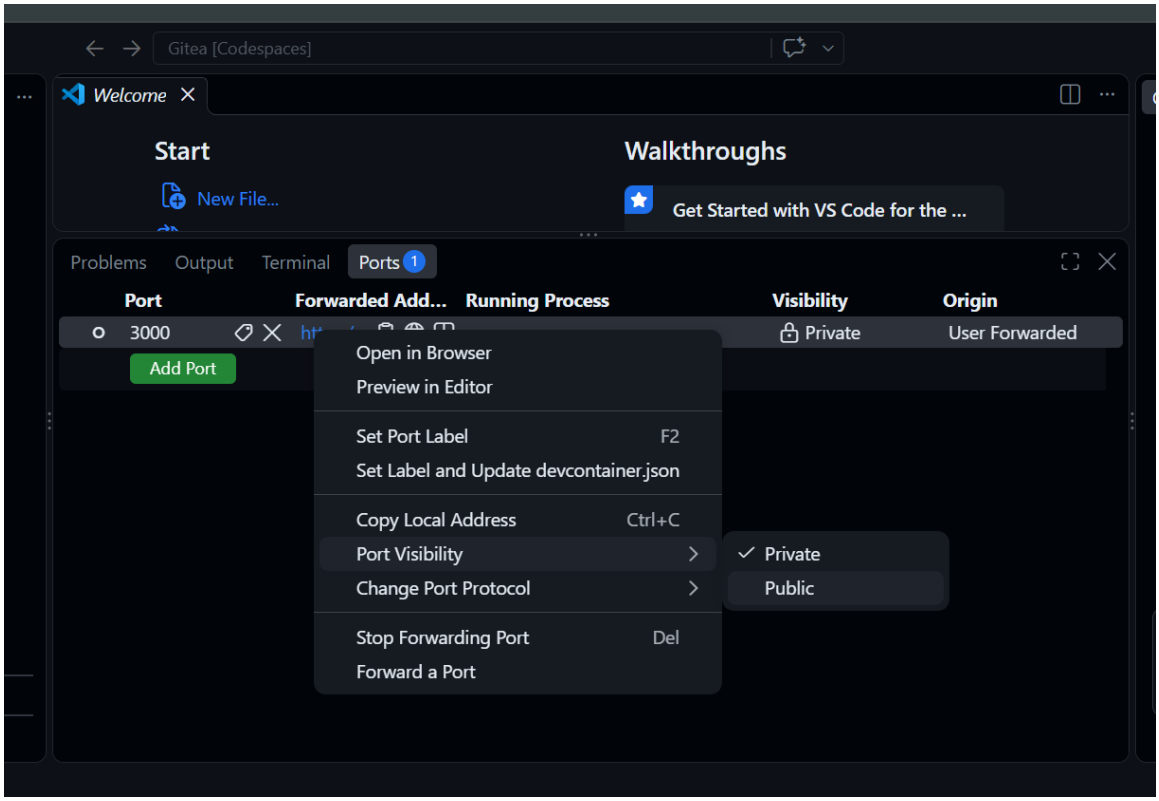


Figure 10.5: Port 3000 menu with Port Visibility options.

Step 6: Changed the port visibility from Private to Public, so that the Gitea page can be opened in the browser.

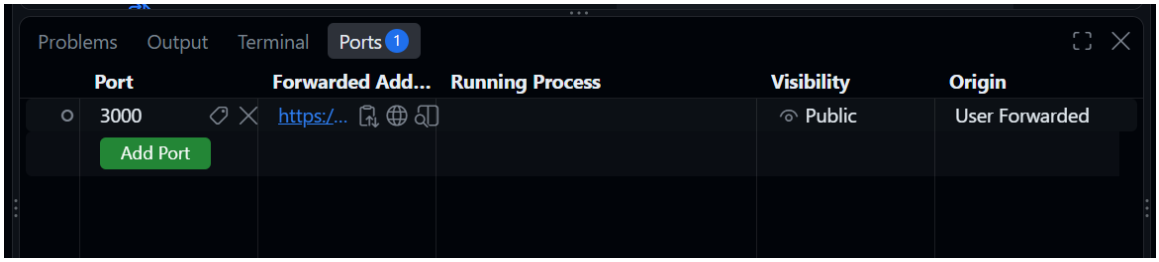


Figure 10.6: Port 3000 is Public.

Step 7: Opened the forwarded port 3000 in the browser. The Gitea initial configuration (installation) page is shown.

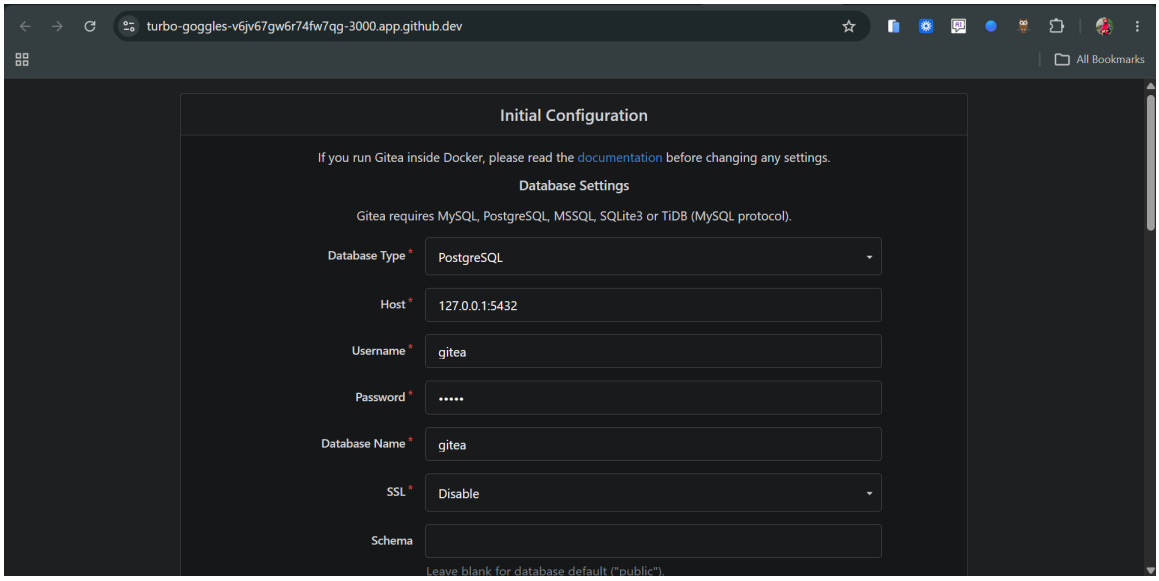


Figure 10.7: Gitea initial configuration page.

Step 8: Filled in the installation form and the Administrator Account Settings (username and email), then installed Gitea.

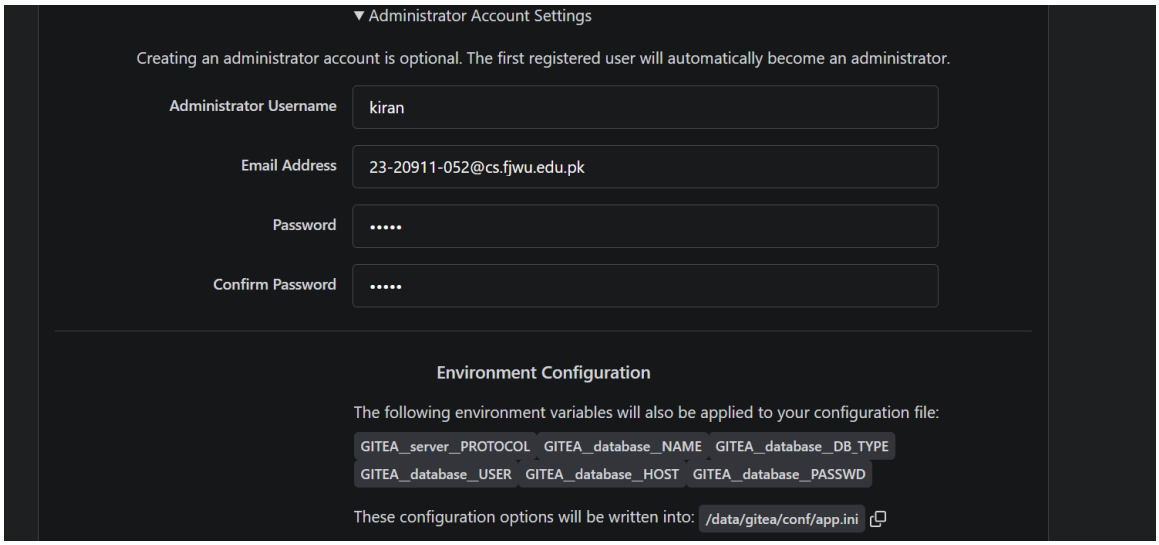


Figure 10.8: Administrator account settings filled in during installation.

Step 9: Logged in to Gitea. The dashboard is shown.

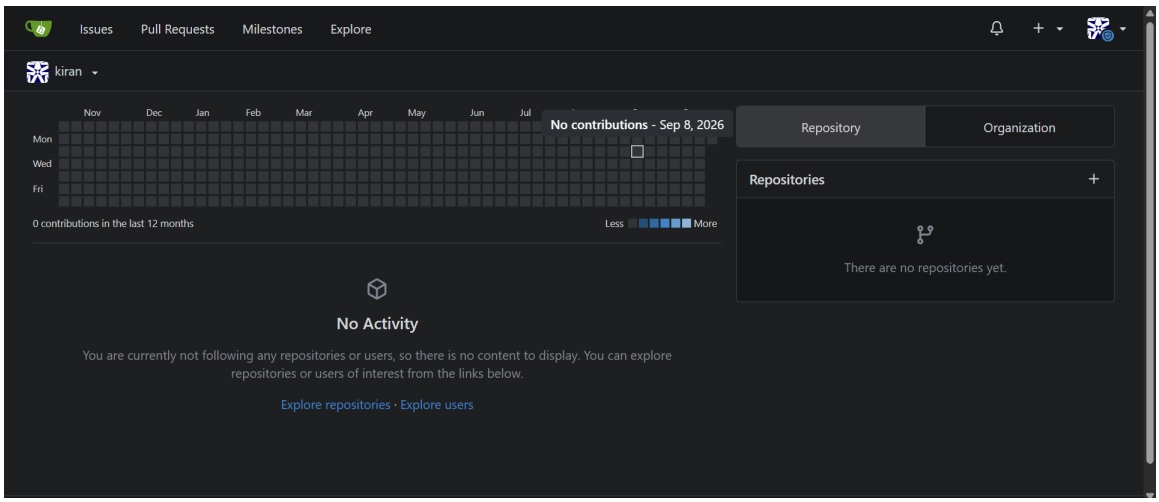


Figure 10.9: Gitea dashboard after logging in.

Step 10: Created a new repository named gitea-test-repo from the + menu.

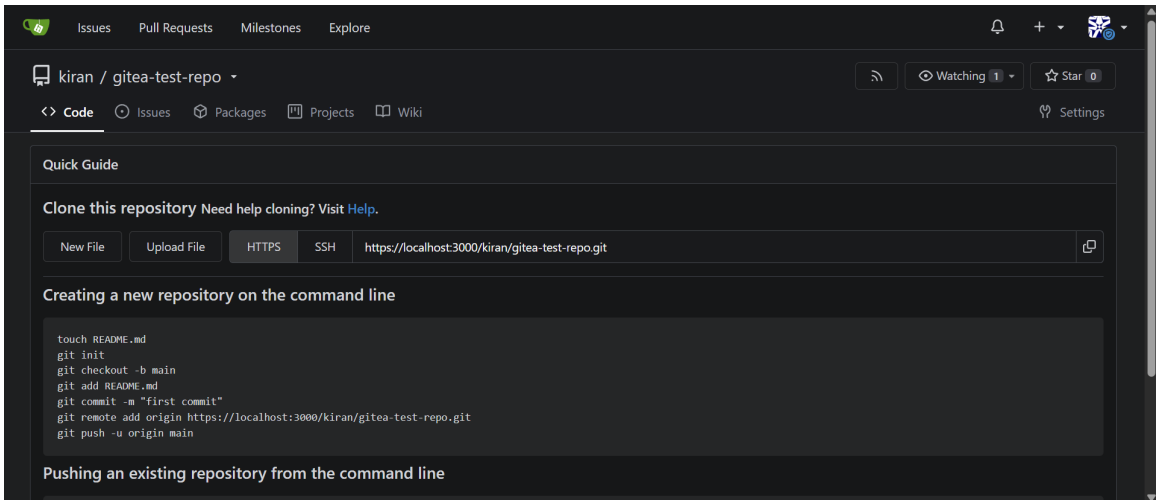


Figure 10.10: New repository gitea-test-repo created in Gitea.

Task 11: Creating a GitHub Pages Portfolio Site

Step 1: Used my existing public portfolio repository (KiranSiddique/Portfolio) instead of creating a new one, because the portfolio was already built and published with GitHub Pages at https://kiransiddique.github.io/Portfolio/.

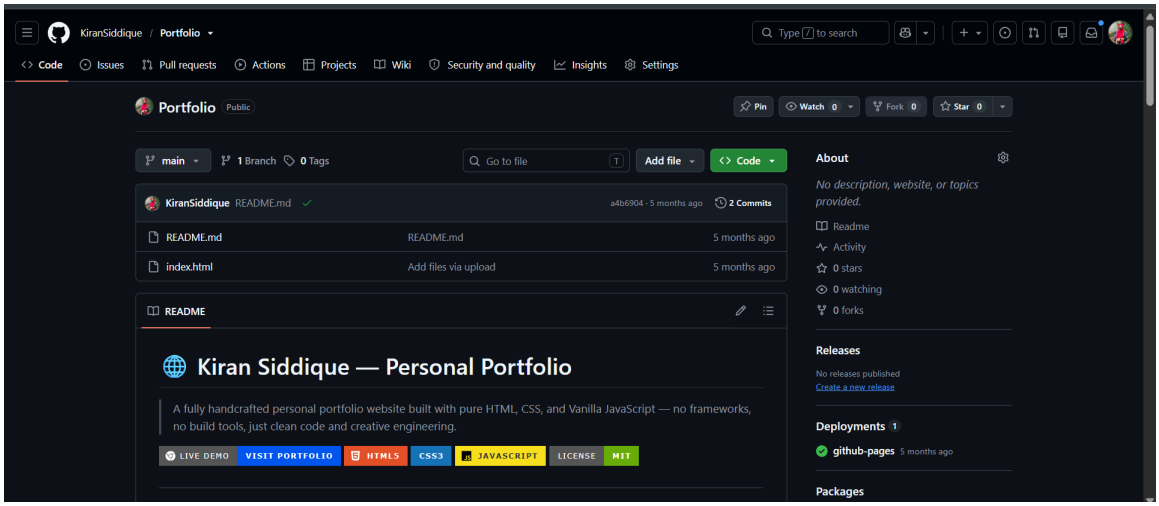


Figure 11.1: Portfolio repository on GitHub.

Step 2: Cloned the portfolio repository into the Ubuntu server and listed its files. The folder contains index.html and README.md.

```
git clone git@github.com:KiranSiddique/Portfolio.git
cd Portfolio
ls -lh
```

```
kiransiddique@ubuntu:~/Portfolio$ ls -lh
total 280K
-rw-rw-r-- 1 kiransiddique kiransiddique 6.1K Oct 18 19:30 README.md
-rw-rw-r-- 1 kiransiddique kiransiddique 271K Oct 18 19:30 index.html
```

Figure 11.2: Local portfolio folder listing with index.html and README.md.

Step 3: Checked the remote URL and the latest commits of the local copy.

```
git remote -v
git log --oneline -3
```

```
kiransiddique@ubuntu:~/Portfolio$ git remote -v
origin  git@github.com:KiranSiddique/Portfolio.git (fetch)
origin  git@github.com:KiranSiddique/Portfolio.git (push)
kiransiddique@ubuntu:~/Portfolio$ git log --oneline -3
a4b6904 (HEAD -> main, origin/main, origin/HEAD) README.md
7a0ad34 Add files via upload
kiransiddique@ubuntu:~/Portfolio$
```

Figure 11.3: Remote URL and the latest commits of the portfolio repository.

Step 4: Opened Settings > Pages on GitHub. The site is live and is published from the main branch.

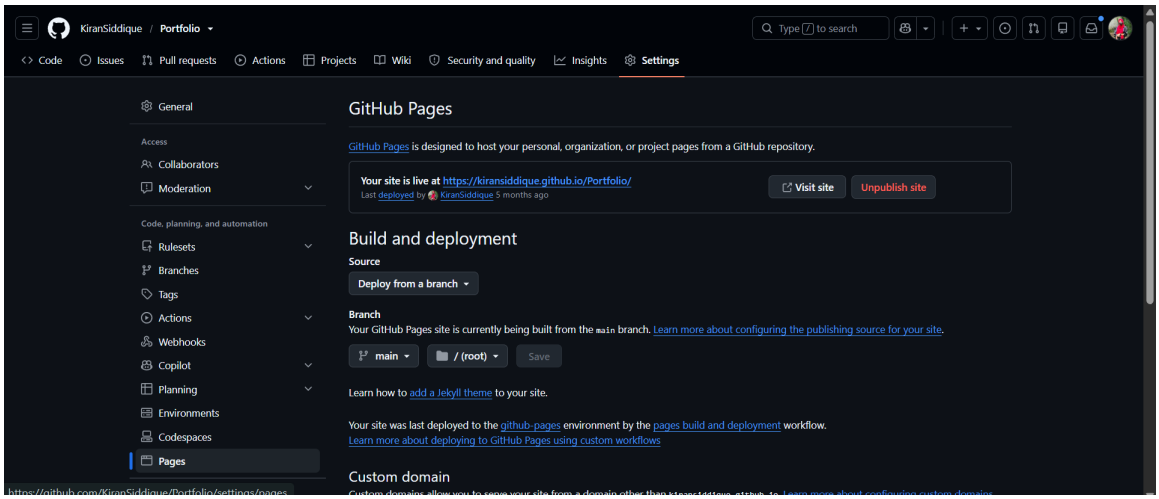


Figure 11.4: GitHub Pages settings, with the site published from the main branch.

Step 5: Opened the live portfolio site in the browser.

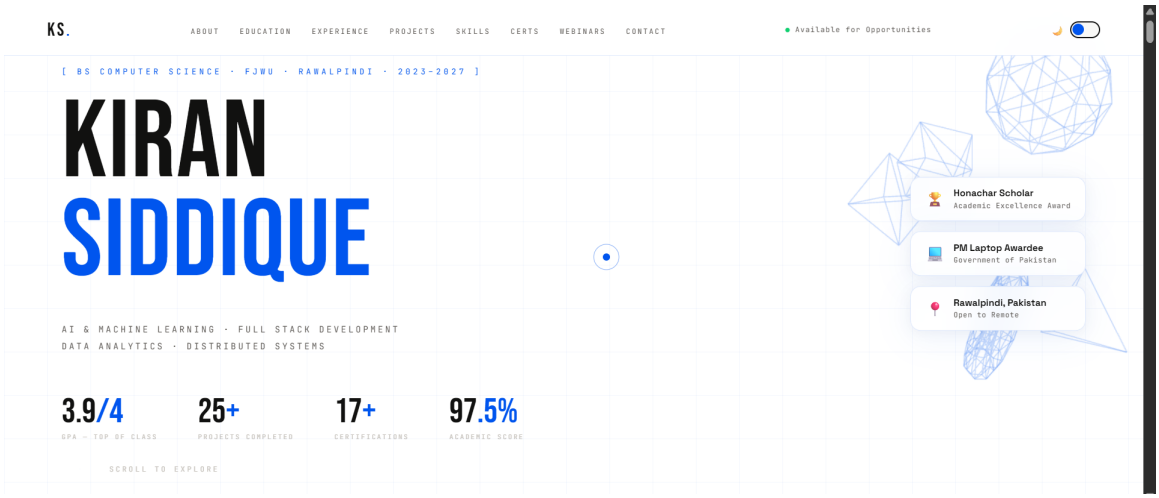


Figure 11.5: Live portfolio site.